Assessment 2 Report

Design Progress and Requirements Specification

Group Name: Aletheia's Zetetes

Product Name: Paedia

CSIT321 - Project 37 - Group 19

University of Wollongong

Prepared for Prof. Jun Yan

Letter to Supervisor

7 May 2026

Prof. Jun Yan
Project Supervisor
University of Wollongong

Dear Prof. Yan,

Please find attached the Assessment 2 design progress report for EduRAG, our course assistant project, with Paedia as the proposed product name. This report brings together the Sprint 2 design baseline, Sprint 3 consultation evidence, and current scope validation into one document for supervisor review and assessment.

The report explains the proposed architecture, key technology decisions, development and deployment environments, and the requirements that will guide implementation. It separates the minimum viable product from additional requirements and stretch goals so that the scope is easier to review. More detailed interface tables, requirement tables, user stories, use case material, and traceability content are included in the appendices.

Where design choices still need confirmation, the report presents them as the team's current design direction rather than approved decisions. We would welcome feedback on the scope, feasibility, and level of difficulty before implementation work becomes more intensive in later sprints.

Yours sincerely,

Group 19
EduRAG Project Team

Executive Summary

EduRAG is a proposed AI-powered course assistant that lets students ask natural language questions about course materials and receive answers grounded in retrieved source content. The project addresses a clear problem in learning: students often struggle to find relevant explanations across slides, notes, and readings when basic keyword search does not match the meaning of their questions.

This report turns the Sprint 2 design outputs, refined through Sprint 3 consultation and prototype evidence, into a supervisor-facing Assessment 2 submission. The current design uses a Python-based modular monolith with LangChain for orchestration, SentenceTransformers for embedding generation, ChromaDB for vector retrieval, PostgreSQL for relational data and session persistence, a Streamlit prototype interface sitting above a lightweight Flask-based WebApplication layer, and Docker Compose for portable multi-container deployment (LangChain, 2026; SentenceTransformers, 2026; Chroma, 2026; PostgreSQL Global Development Group, 2026; Streamlit, 2026; Flask, 2026; Docker Inc., 2026).

The requirements specification defines a focused MVP centred on authentication, document ingestion, semantic retrieval, grounded answer generation, source citation, asynchronous indexing, and multi-course isolation. Additional requirements extend the product with multi-turn sessions, admin monitoring features, document management, and rate limiting, while stretch goals such as export, summarisation, and follow-up suggestions are kept separate to control scope.

The Sprint 2 design baseline is technically strong, and the Sprint 3 evidence strengthens its feasibility for the Assessment 2 submission. Student, Lecturer, and Admin use case material is now grouped more consistently, terminology is standardised around ChromaDB and PostgreSQL, and wording that could imply prior supervisor approval has been removed. The aim is to make the document clearer and easier to review before implementation accelerates.

Table of Contents

Letter to Supervisor 2
Executive Summary 3
Table of Contents 4
Introduction 7
Market Analysis and Project Rationale 7
Branding and Presentation Direction 10
Initial Interface Design Direction 11
Design Methodology 13
Supervisor Consultation, Scope Validation, and Meeting Evidence 14
Development Environment 15
Deployment Environment 16
Tools and Technology Decisions 17
Requirements Analysis 18
- Acceptance Criteria and Validation Plan 19
- Representative Base Requirements 20
- Representative Additional Requirements 20
- Representative Stretch Goals 20
- Explicit Exclusions 20

Creativity, Innovation, and Project Difficulty 20
- Project Difficulty Justification Matrix 21

Architecture Summary 21
- Module Responsibilities 22
- RAG Pipeline 24
- RAG Evaluation Strategy 25
- Technical Risk and Responsible Use Controls 26
- Deployment Topology 27

Conclusion 29
References 29
Glossary 31
Appendix A. Technology Decision Matrix 33

Appendix B. Module Interface Specifications 35
Module 1 - DocumentProcessor 35
Module 2 - VectorRetriever 35
Module 3 - LLMResponseGeneration 36
Module 4 - WebApplication 37
Inter-module communication summary 38
Appendix C. Full Functional Requirements 39
Appendix D. Full Non-Functional Requirements 47
Appendix E. User Stories and Use Case Descriptions/Diagram 52
Student Use Cases 56
UC-01 Select Course Context 58
UC-02 Submit Question 58
UC-03 Receive AI-Generated Response 59
UC-04 View Source References 60
UC-05 Continue Multi-turn Conversation 61
UC-06 Resume Saved Session 62
UC-07 Review Conversation History 63
UC-08 View Confidence Indicator 64
Lecturer Use Cases 65
UC-09 Upload Course Materials 66
UC-10 Organise Materials by Topic 67
UC-11 Update Course Materials 68
UC-12 Delete Course Materials 69
UC-13 Preview Document Chunks 70
UC-14 View Student Query Logs 71
UC-15 Flag Inaccurate Responses 72
Admin Use Cases 74
UC-16 Manage User Accounts 75
UC-17 Assign Lecturers to Courses 76
UC-18 Reset User Passwords 77
UC-19 Configure LLM Settings 78
UC-20 Monitor System Metrics 79

UC-21 Export Audit Logs 80
UC-22 Trigger Vector Re-indexing 81
Appendix F. Traceability Matrix 82
Appendix G. Additional Diagrams 85
G1. Entity Relationship Diagram (ERD) 85
G2. Data Flow Diagram (DFD Level 1) 86
G3. Sequence Diagram (Upload exception and recovery) 87
Appendix H. Initial Interface Wireframes 88
H1. Login and Role Entry 88
H2. Student Q&A Interface 89
H3. Lecturer Document Upload 90
H4. Admin Dashboard 91
H5. Session History 92

Introduction

EduRAG is a capstone project in CSIT321 that applies Retrieval-Augmented Generation (RAG) to the problem of course-specific knowledge access. RAG was introduced by Lewis et al. (2020) as a technique that combines a dense retrieval step with a generative language model: the system first retrieves relevant passages from a knowledge base, then conditions its generated answer on those passages. Applied to an educational context, this means students can ask natural language questions and receive answers that are grounded in the actual materials their lecturer has uploaded, rather than in the model's general training knowledge.

The system is not intended to replace lecturers, learning management systems, or formal teaching materials. Instead, it is designed as a support tool that helps students navigate existing course resources through conversational search. The key design principle is transparency: every answer either carries a source citation traceable back to a specific course document, or states that the answer cannot be verified from the available course materials. This distinction directly addresses the hallucination risk that makes ungrounded language models unsuitable for academic support.

Assessment 2 requires a professional report that explains the project's development direction, requirements baseline, and design methodology. This report reorganises the Sprint 2 design baseline and later consultation evidence into a structured format for supervisor review, keeping detailed technical artefacts in the appendices. The report covers target users, development and deployment environments, technology decisions, system architecture, and full requirements analysis. It clearly separates what the minimum viable product must deliver, what improvements are desirable if time permits, and which features are out of scope.

Market Analysis and Project Rationale

Market context and scale. Paedia is aimed at a large, growing, and digitally dependent higher education environment, not a one-off classroom convenience problem. Australian higher education had 1,676,077 enrolled students in 2024, up 4.7% from 2023, so even a small improvement in how students retrieve course-specific information can affect a substantial learner population. For EduRAG, this market size matters because the product addresses a repeated study workflow: students ask questions, locate evidence, check source material, and prepare assessment work across several subjects.

Why now. The timing matters because student AI use has already moved faster than most institutional controls. Grand View Research (2024) projects the global AI-in-education market at USD 5.88 billion in 2024 and a 31.2% compound annual growth rate from 2025 to 2030. Chegg's 2025 global student survey found that 80% of surveyed students had used generative AI to support their studies, while 53% of GenAI users were concerned about receiving incorrect or inaccurate information from tools not built specifically for education. The opportunity is therefore not to convince students to try AI, but to give them a safer course-grounded option when they already expect AI-supported study help.

Student problem. Students face a practical information access problem. Course knowledge is distributed across lecture slides, PDFs, weekly readings, LMS announcements, rubrics, tutorial notes, and recorded sessions. Generic search is often too broad, LMS search is document-oriented rather than question-oriented, and general-purpose chatbots may answer confidently without being constrained to the subject material. This creates three recurring pain points: slow retrieval, uncertainty about whether an answer is based on the course, and weak traceability back to evidence.

Trust and accuracy gap. The strongest rationale for Paedia is not novelty; it is trust. HEPI reported that 92% of UK students had used AI in some form, while 51% identified false results or hallucinations as a factor putting them off using AI. That combination is important: students are using AI despite doubts about reliability. Paedia responds by narrowing the answer space to uploaded course materials, returning citations, preserving course isolation, and exposing confidence cues. These controls make the product more appropriate for academic study than an unconstrained chatbot.

Evidence area	**Real figure / finding**	**Relevance to Paedia**
Australian higher education scale	1,676,077 students enrolled at higher education providers in 2024; up 4.7% from 2023.	Shows the addressable learner population is large enough for course-level AI support to matter beyond a single class.
AI-in-education growth	Global AI-in-education market estimated at USD 5.88 billion in 2024, with 31.2% CAGR forecast from 2025 to 2030.	Confirms the market is moving toward AI-supported learning tools and institutions need defensible, governed options.
Student AI adoption	Chegg reported that 80% of surveyed students had used generative AI to support their studies, while 53% of GenAI users were concerned about incorrect or inaccurate information from non-education-specific tools.	Indicates Paedia meets an existing behaviour rather than creating a new one.
Trust gap	HEPI reported that 92% of UK students had used AI in some form, and 51% said false results or hallucinations put them off using AI.	Supports the need for citations, source grounding, confidence indicators, and course-specific retrieval.
Institutional governance	Universities increasingly	Supports Paedia's design

	require transparent and responsible AI use in assessment contexts.	focus on traceability, scoped retrieval, auditability, and role-based control.

Table 1. Market evidence supporting Paedia's rationale.

Target users. Paedia has two primary user groups and one enabling user group. Students need fast, cited explanations grounded in their subjects. Lecturers need a way to provide approved material to the retrieval system and review whether answers reflect the course. Administrators need account, course, and system controls so the tool can operate within an institutional environment.

User group	**Current pain point**	**Paedia value proposition**
Students	Study materials are scattered and generic AI may hallucinate or cite weak sources.	Ask natural-language questions and receive concise, cited answers grounded in enrolled-course materials.
Lecturers	Students repeatedly ask clarification questions and may rely on unsupported external AI answers.	Upload and manage course documents, support consistent responses, and review inaccurate answers through a feedback workflow.
Administrators	Institutional AI tools require control over users, access, configuration, and monitoring.	Manage accounts, course access, system metrics, audit logs, and re-indexing in a controlled environment.

Table 2. Target users and Paedia value proposition.

Competitive positioning. Paedia is positioned between three existing alternatives: generic AI chatbots, LMS search tools, and broad study platforms. Generic AI tools are flexible but weakly grounded in a specific subject. LMS search can find files but usually cannot synthesize an answer. Study platforms provide explanations but are not aligned to a university's exact weekly content. Paedia differentiates itself through course-scoped retrieval, source citations, lecturer-controlled content, and an assessment-aware design.

Alternative	**Strength**	**Limitation**	**Paedia differentiation**
Generic chatbots	Fast conversational answers and	Not constrained to local course materials and may	Course-grounded RAG, citations, confidence cues,

	broad knowledge coverage.	produce unsupported answers.	and fallback when evidence is insufficient.
LMS document search	Already connected to course files and institutional access.	Keyword search does not reliably answer natural-language study questions.	Question-answering layer over course documents with retrieval evidence.
General study platforms	Useful explanations and practice support.	May not match the exact subject, lecturer expectations, or assessment context.	Uses lecturer-provided material and course-specific indexing.
Manual lecturer support	High trust and direct alignment with course intent.	Limited availability and repetitive clarification workload.	Handles repeatable evidence-based questions while preserving lecturer oversight.

Table 3. Competitive positioning against key alternatives.

Project rationale. Paedia should exist because it addresses a clear gap between student behaviour and institutional trust requirements. Students are already using AI for study, but current options do not reliably prove that answers come from approved course sources. A RAG-based architecture is suitable because it separates retrieval from generation: the system first searches indexed course material, then generates an answer using retrieved evidence. This allows Paedia to deliver the convenience students expect while supporting the transparency, traceability, and governance that universities require.

Branding and Presentation Direction

Assessment 2 asks the team to begin applying a consistent project identity from this stage onward. For this report, the group name is `Aletheia's Zetetes` and the proposed product name is `Paedia`. The branding approach is still lightweight, but it is now explicit and consistent across the report.



Figure 1. Aletheia's Zetetes group logo used for the current report identity.

Branding element	Current direction
Group name	Aletheia's Zetetes
Product name	Paedia
Project name	EduRAG
Identity style	Academic and professional presentation with the existing group logo and a simple product name
Primary colours	Navy for major headings, teal for secondary emphasis, gold for tertiary emphasis
Tone	Professional, educational, and technically grounded
Future carry-over	Use the same group name, product name, colour treatment, and figure styling in Sprint 4 presentation assets and the prototype UI

This level of branding is appropriate for the project's current maturity. It shows clear identity choices without spending too much effort on marketing assets before the core design and requirements have been validated.

Initial Interface Design Direction

The initial interface design translates the requirements baseline into visible user workflows without changing the Assessment 2 scope. The Sprint 3 UI package prepared by Anmoldeep Singh uses Paedia's navy, teal, and gold visual system to show how Students, Lecturers, and Administrators would interact with the same RAG-backed platform.

The design is intentionally requirements-led. It does not treat the interface as decoration; each screen demonstrates how a functional or non-functional requirement can be experienced by a real user.

Interface element	Requirement/design evidence
Student Q&A view with active course context	Supports natural-language questions, course_id-scoped retrieval, multi-turn sessions, and source-backed answers.
Source citation chips and confidence indicator	Makes answer grounding visible so students can verify course material references and judge retrieval confidence.
Fallback label for ungrounded answers	Protects academic trust by clearly stating when no grounded course-material answer can be verified, rather than presenting unsupported model knowledge.
Lecturer upload and indexing workflow	Shows how uploaded course files move into asynchronous processing, status tracking, and ChromaDB indexing.
Role-based navigation and admin dashboard	Separates Student, Lecturer, and Admin capabilities while supporting audit, metrics, and system monitoring needs.
Accessible colour and component rules	Uses consistent contrast, readable typography, status colours, and predictable controls to support usability requirements.

The three representative wireframes below show the main interaction paths: asking a grounded question, uploading course materials, and monitoring the system as an administrator. The complete five-screen wireframe set is included in Appendix H.

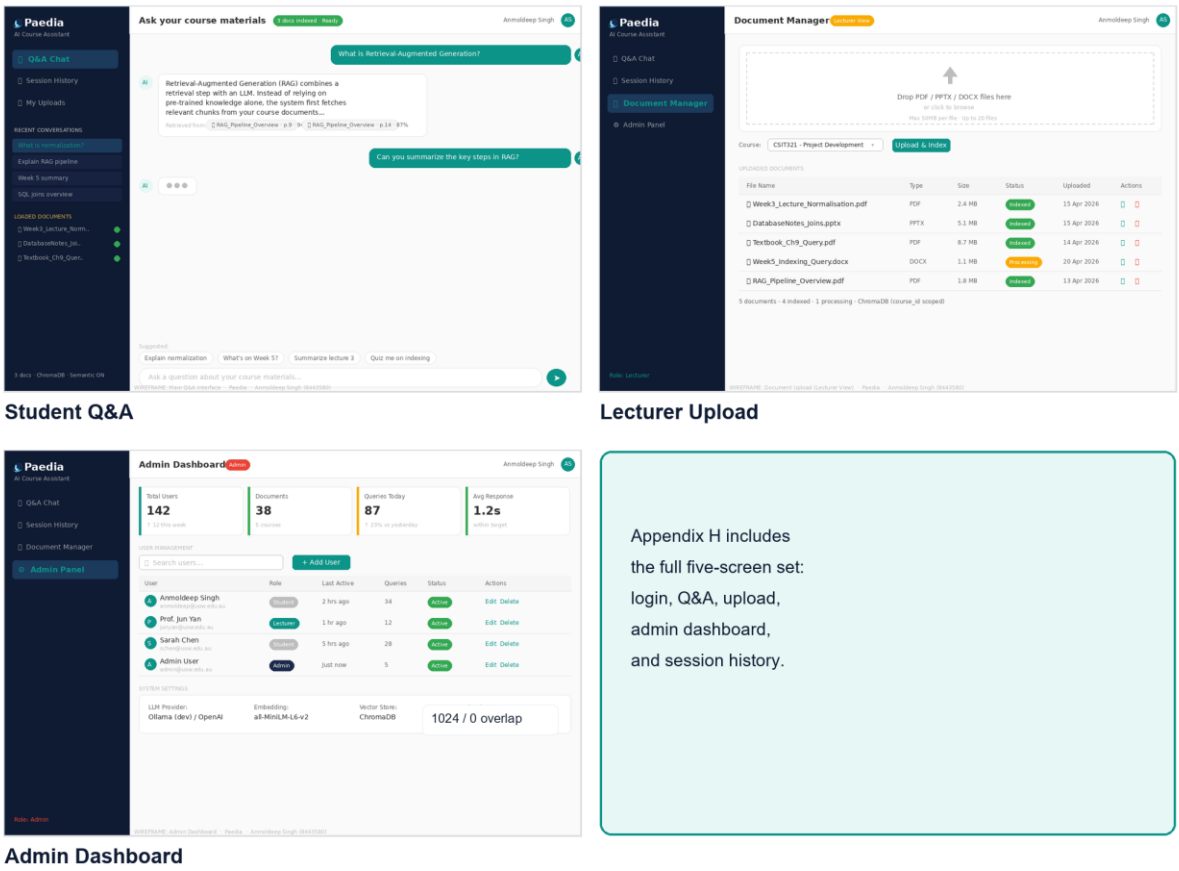


Figure 2. Interface contact sheet. Representative wireframes for Student Q&A, Lecturer Upload, and Admin Dashboard.

Design Methodology

The project follows an Agile/Scrum approach with two-week sprints (Schwaber and Sutherland, 2020), with Sprint 2 acting as the main design-definition phase. This approach suits the project because there is still technical uncertainty in areas such as document ingestion quality, embedding performance, and multi-provider LLM integration. An iterative process helps the team discover these risks early while still keeping a clear planning rhythm.

1. User roles and likely interactions were identified first through stakeholder analysis and user stories.
2. Those interactions were converted into use cases, then formalised into functional requirements using MoSCoW prioritisation.
3. Quality, security, reliability, and maintainability targets were captured as measurable non-functional requirements.
4. Architecture, interfaces, and deployment decisions were then selected to satisfy the resulting requirement set.

5. Traceability was maintained so that requirements can be mapped back to stakeholder needs rather than appearing as isolated technical preferences.

This methodology is also appropriate for Assessment 2 because the report is expected to show a requirements specification shaped by the team's development process. The MoSCoW model fits the sprint plan well: Must-have requirements define the MVP, Should-have requirements support later enrichment, and Could-have requirements remain stretch work.

Supervisor Consultation, Scope Validation, and Meeting Evidence

The project scope was refined through sponsor meetings, coordinator check-ins, and internal sprint planning rather than treated as a fixed initial brief. The Sprint 3 meeting report set supplied by the team records six consultation and planning moments from 01 April to 01 May 2026; the Sprint 3/Sprint 4 evidence was documented by Tung Phuong Anh Huynh and gives a stronger chronological basis for explaining how EduRAG moved from concept to validated design direction.

The table below consolidates the previous supervisor-consultation narrative and evidence map into one evidence trail. It separates validated guidance from planned Sprint 4 work so the report does not overclaim results that still require implementation or benchmark evidence.

Meeting / date	Evidence from meeting report	Scope or design decision	How this report reflects it
Meeting 1 - Sponsor, 01 Apr 2026	Project description treated as a starting point; team asked to clarify product thesis, user need, competitors, dependencies, milestones, and delivery process.	The product should be justified through requirements and market/user analysis, not copied directly from the initial brief.	Introduction, market rationale, requirements grouping, design methodology, and milestones explain why Paedia is needed and how the team will deliver it.
Meeting 2 - Sponsor, 15 Apr 2026	Modular monolith, ChromaDB, PostgreSQL, Ollama, Streamlit MVP, and React stretch path were confirmed. Course isolation and source-referenced answers were identified as final-demo differentiators.	MVP should focus on course-scoped RAG, cited answers, and a practical Streamlit-first interface; advanced frontend work remains stretch.	Architecture, deployment, requirements, and RAG sections foreground course_id filtering, citations, role separation, Streamlit/Flask delivery, and React as stretch.
Meeting 3 - Coordinator check-in, 15 Apr 2026	Coordinator check-in confirmed progress, surfaced attendance/coordination concerns, and recorded no other blockers at that point.	Progress tracking and blocker escalation were required controls for Sprint 3 delivery.	Design methodology and validation discussion present the report as sprint-shaped evidence, not a static design document.
Meeting 4 - Internal group, 15 Apr 2026	Team confirmed the end-to-end RAG PoC as Sprint 3 success criterion, Docker verification, 31-question test corpus, sample materials, API specification, and UI/backend integration checkpoint.	Sprint 3 evidence should separate demonstrated PoC work from integration and evaluation still planned for Sprint 4.	RAG Pipeline, RAG Evaluation Strategy, acceptance criteria, and appendices distinguish PoC evidence, 31-query benchmarking, parser coverage, and integration validation.
Meeting 5 - Sponsor review, 01 May 2026	Live RAG PoC and Streamlit UI were demonstrated. Public online course materials were accepted for demo-stage evidence; single ChromaDB course_id filtering was considered reasonable; A2 structure/length was acceptable if clear.	Current A2 scope is appropriate, while richer subject-specific materials and broader feature expansion remain future work after core stability.	Report keeps demo-corpus constraints visible, preserves the accepted 1024/0 chunking baseline, and marks richer test data, API integration, and expanded validation as Sprint 4 or later work.
Meeting 6 - Sprint 4 internal planning, 01 May 2026	Team identified A2 submission as the critical path, prioritising Appendix G/proofread, missing Sprint 3 meeting deliverables, Moodle readiness, and Flask API wrapper work.	A2 readiness and evidence consistency should be resolved before broader implementation acceleration.	Final report consolidates consultation evidence, aligns Appendix G and interface/architecture evidence, and positions Flask WebApplication/API work as Sprint 4 direction.

Taken together, the meeting reports validate the direction of Paedia as a course-specific RAG assistant rather than a general AI chatbot. The most important product evidence is that students need grounded answers from course materials, while the most important technical evidence is that the team has a feasible path for course isolation, source citation, role separation, and local demonstration. The report therefore treats the Streamlit/Flask MVP, ChromaDB/PostgreSQL storage split, Ollama-first development path, and 31-query evaluation plan as the current design baseline, while keeping React

migration, richer personalisation, broader material types, and deeper subject-specific validation as later-sprint expansion.

Validation status. Sponsor and coordinator meetings support the design direction, scope boundaries, and readiness priorities. They do not prove every Sprint 4 feature has already been implemented. Items such as Flask endpoint completion, full UI/backend integration, rerun benchmark results, and richer subject-specific corpus validation remain implementation and validation tasks that must be evidenced separately.

Development Environment

The proposed development environment is a Python-centred local setup designed to reduce onboarding friction for a student team while still reflecting professional engineering practice. Core application logic is expected to live in a single repository with Git-based version control, structured branching, and incremental integration through pull requests. For the MVP, the team plans to use a Streamlit prototype interface on top of a lightweight Flask-based WebApplication layer so the UI can evolve quickly without losing a clean backend boundary.

Area	**Proposed environment**
Programming language	Python for all core prototype modules
RAG orchestration	LangChain as the selected framework; LlamaIndex retained as a considered alternative
User interface	Streamlit prototype UI for rapid MVP development
Application service	Flask-based WebApplication layer for authentication, query handling, and document actions
Embeddings	SentenceTransformers with all-MiniLM-L6-v2
Vector store	ChromaDB, initially local and persistent
Relational data	PostgreSQL for users, sessions, and audit data
Local model option	Ollama for cost-controlled development and local testing
Project coordination	Sprint-based task planning and meeting documentation

This environment fits the team's educational constraints well. Developers can work productively with one language, one main application codebase, and a small number of infrastructure services. At the same time, the chosen stack still exposes the team to realistic patterns such as vector retrieval, structured persistence, role-based access control, containerisation, and evaluation-oriented requirements.

Deployment Environment

EduRAG is designed to run as a multi-container Docker application on a single host for development and demonstration. This environment is intentionally pragmatic: it is portable enough to run on team laptops or a dedicated demo machine, but not so complex that the project becomes dependent on cloud infrastructure before the core pipeline is validated.

The proposed deployment uses Docker Compose to run the application container, PostgreSQL container, and ChromaDB container on a single host. Ollama is treated as the primary local LLM service during development and PoC demonstration, while the same generation module can switch to the OpenAI API through environment configuration if a controlled demonstration requires it. Persistent Docker volumes store uploaded files, relational data, and vector indexes so restarts do not erase key system state (Docker Inc., 2026; PostgreSQL Global Development Group, 2026; Chroma, 2026; Ollama, 2026).

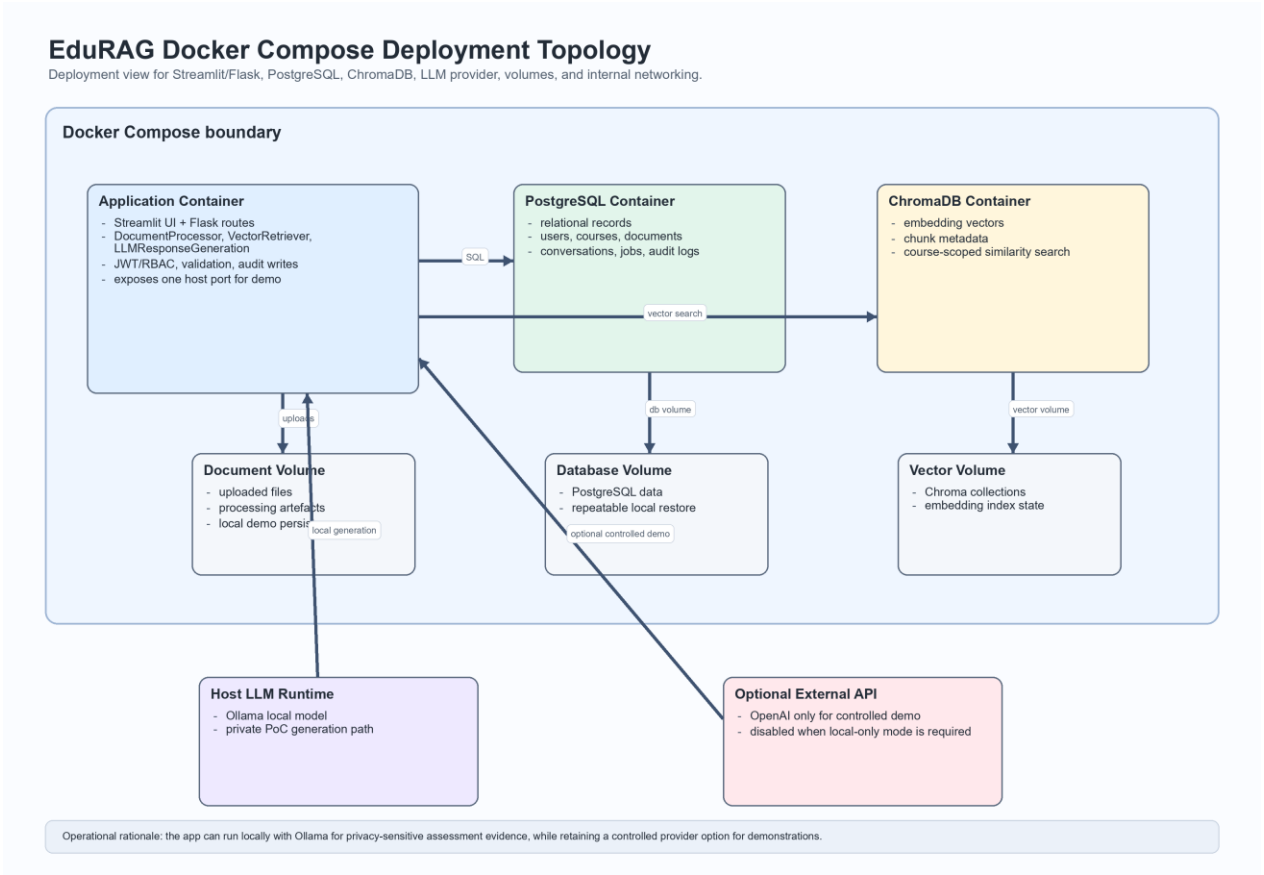


Figure 3. Docker Compose deployment topology and service boundaries.

This deployment strategy also preserves a clear migration path. If the MVP succeeds early, the Streamlit prototype interface can later be replaced by a dedicated React frontend and a FastAPI service layer without changing the document processor, retriever, or response-generation modules. That future option is one reason the modular monolith pattern was selected.

Tools and Technology Decisions

The current technology stack was selected by comparing alternatives against project scope, team capability, prototype cost, and operational simplicity. The resulting decisions are intentionally conservative: where two options were technically viable, the design favoured the option that reduced integration overhead while still providing clear educational value.

Component	Decision	Primary Rationale
LLM Provider	Ollama primary + OpenAI optional controlled-demo fallback	Cost-controlled local development and PoC demonstration, with hosted-model option only if demo quality or availability requires it.
Vector Database	ChromaDB	Native course_id metadata filtering; built-in persistence
Embedding Model	all-MiniLM-L6-v2	Lightweight 384-dim; fast; well-supported
Prototype stack (MVP)	Streamlit UI + Flask WebApplication layer (Sprint 4 API direction)	Fast feedback for the prototype while preserving a clean application boundary; Sprint 3 used a generic Python app service concept, while Sprint 4 tasks specify Flask route implementation.
Web Framework (Stretch)	FastAPI + React	Production-grade migration path
Relational Database	PostgreSQL	RBAC support; JSONB for conversation history
Authentication	JWT + RBAC (3 roles)	Industry standard; works with both MVP and stretch
Deployment	Docker + docker-compose	Multi-container; portable; professional
Architecture	Modular Monolith	Microservices discipline without operational overhead
Document Formats	PDF + PPTX + DOCX	Broad coverage of course materials
Course Scoping	Multi-course (course_id)	Real-world ready; matches Module 2 spec
Conversation Mode	Multi-turn + saved sessions	Persistent history via PostgreSQL JSONB

LangChain is the selected orchestration framework because it provides a practical abstraction over prompt templates, retrieval chains, and multiple LLM providers, which is useful for using Ollama locally while retaining an OpenAI API option for a controlled demonstration if required (LangChain, 2026; OpenAI, 2026; Ollama, 2026). LlamaIndex remains a credible alternative and was considered during evaluation because of its strong support for context augmentation over private data, but it was not chosen as the primary framework for the MVP (LlamaIndex, 2026).

The most important Sprint 2 refinement was the shift from FAISS to ChromaDB and the formal commitment to PostgreSQL. This change is defensible because course-specific retrieval, session

persistence, JWT-based role handling, and audit-oriented relational data are core parts of the proposed product rather than optional extras.

Hybrid retrieval and reranking remain plausible future enhancements, but they are intentionally deferred until the single-vector baseline has been benchmarked on real course materials. BEIR shows that lexical retrieval and reranking can be strong baselines, yet they also introduce extra pipeline and evaluation complexity that is difficult to justify before the MVP corpus and curated query set have stabilised (Thakur et al., 2021).

Requirements Analysis

The requirements specification is the main focus of Assessment 2, so the report explicitly distinguishes between the minimum viable product, additional requirements, and stretch goals. This structure is derived from the Sprint 2 MoSCoW prioritisation and is intended to give the supervisor a clear view of project difficulty, delivery realism, and scope boundaries.

Delivery layer	Source priority	What it means in this project	Examples
Base requirements	Must have	Capabilities required for EduRAG to function as a usable RAG assistant	authentication, PDF upload, chunking, embeddings, ChromaDB indexing, semantic retrieval, grounded answers, citations, async indexing, multi-course isolation
Additional requirements	Should have	Important quality and management features that strengthen the MVP if delivery remains on track	multi-turn sessions, saved conversations, admin dashboard, audit log review, rate limiting, document management, confidence scoring, retrieval benchmarks
Stretch goals	Could have	Enhancements that improve polish or usability but are not necessary for the prototype to demonstrate value	conversation export, topic summarisation, follow-up question suggestions, dark mode, usage analytics export

The base requirement set is appropriately challenging for a capstone prototype because it requires the team to integrate document processing, semantic retrieval, grounded LLM

generation, security controls, and basic observability into one coherent system. It is also contained enough to remain feasible within the annual session.

Acceptance Criteria and Validation Plan

The following plan summarises how the current requirement set should be validated during Sprint 4. It is a design-stage validation plan, not a claim that all checks have already been executed. Detailed FR and NFR definitions remain in Appendices C and D; this matrix groups the most important acceptance areas for integration testing and A3 evidence capture.

Validation area	Related requirement focus	Acceptance direction	Planned evidence
Source citations	Grounded answers and citation display	Grounded answers should expose source references such as filename, page number, chunk metadata, and similarity score where available.	Chat screenshot and response payload showing source_references.
Confidence indicator	Retrieval confidence and user trust	Confidence should be visibly derived from retrieval strength, not from the model tone alone.	High, medium, and low confidence examples using the design-system colours.
Course selection	Course-scoped retrieval	Changing the active course should change course_id and prevent cross-course retrieval leakage.	Same query tested in two course contexts with different scoped evidence.
Upload and indexing status	Document upload, parsing, chunking, and indexing	Lecturer uploads should show pending, processing, indexed, or failed states for PDF/PPTX/DOCX inputs.	Upload flow screenshots and indexing status response.
Role-based navigation	Student, Lecturer, and Admin access separation	Each role should see only the relevant navigation items and protected views.	Role walkthrough screenshots or fixture-based UI evidence.
Fallback label	Unsupported or weak-retrieval answers	Unsupported questions should show the approved fallback label and avoid false course-grounded citations.	Off-topic query result with is_fallback=true.
Accessibility and colour consistency	Basic accessibility and visual consistency	Text contrast, focus indicators, status colours, and UI states should follow the design system.	Manual accessibility checklist and visual consistency screenshots.

Representative Base Requirements

- Students can submit natural language questions and receive grounded answers with source citations.
- Lecturers can upload course materials for asynchronous processing and indexing.
- The system stores embeddings in ChromaDB scoped by course_id to avoid cross-course leakage.
- Authentication and role-based access control separate Student, Lecturer, and Admin capabilities.
- The retrieval and generation path is measurable through latency, citation rate, precision, recall, and hallucination targets.

Representative Additional Requirements

- Conversation sessions persist across logins so users can continue prior discussions.
- Administrators can inspect metrics and audit logs to review system behaviour.
- Rate limiting and confidence indicators improve robustness and user trust.

Representative Stretch Goals

- Conversation export to PDF for revision or record-keeping.
- Topic summarisation and suggested follow-up questions to deepen learning support.
- Visual enhancements such as dark mode once core functionality is stable.

Explicit Exclusions

The report also preserves the Sprint 2 Won't-have boundary to prevent scope creep. The current prototype does not include live lecture transcription, a multi-language user interface, LMS integration, native mobile applications, or automated grading.

Creativity, Innovation, and Project Difficulty

EduRAG is innovative in the context of this subject because it combines retrieval, generation, course scoping, and multi-role interaction into a single educational assistant rather than delivering only a chatbot, only a search tool, or only a document repository. The project is not attempting novelty for its own sake; its innovation lies in how established AI techniques are integrated into a realistic educational workflow.

Relative to simpler alternatives, the project is more challenging because it must solve several linked problems at once: reliable extraction from course documents, embedding and vector retrieval, grounded prompt construction, answer traceability through citations, role-based security, and persistent session management. This makes the project difficult enough to justify capstone-level assessment while still remaining implementable within the annual sprint structure.

Assessment-facing innovation comparison. A generic chatbot would be easier to build but would not prove that answers come from approved course sources. An LMS keyword search

would be easier to govern but would not synthesise explanations. A static document repository would store materials but would not help students ask contextual questions. Paedia is more valuable because it combines these paths into one course-scoped, cited, role-aware study workflow; it is more difficult because each answer must preserve retrieval quality, citation traceability, access control, and responsible fallback behaviour at the same time.

Project Difficulty Justification Matrix

The difficulty of Paedia comes from coordinating multiple technical constraints in one coherent prototype. The matrix below makes the difficulty explicit rather than relying only on a general statement that the project uses RAG.

Difficulty area	Why it is difficult	Design response in Paedia
Hallucination control	A fluent answer can be wrong if it is not grounded in course material.	Use course-scoped retrieval, citation display, safer fallback/refusal wording, and planned groundedness checks.
Retrieval quality	Useful answers depend on chunking, embeddings, top-k retrieval, similarity thresholds, and test coverage.	Adopt chunk_size=1024, chunk_overlap=0, top_k=5, threshold=0.65, all-MiniLM-L6-v2, and a 31-question labelled corpus for Sprint 4 validation.
Multi-role access	Students, lecturers, and admins need different permissions while sharing the same system.	Separate role-based navigation, protected endpoints, course enrolment scope, and admin-only audit/metrics views.
Persistence	The system must preserve users, courses, documents, sessions, messages, audit events, and indexing jobs across restarts.	Use PostgreSQL for relational state and ChromaDB for vector state, with Docker volumes for prototype portability.
Document parsing	PDF, PPTX, and DOCX files contain different structures and can fail in different ways.	Preserve source metadata and slide/document structure where possible while keeping the accepted chunking baseline consistent.
Auditability	Educational AI needs traceability for trust, debugging, and accountability.	Store source references, session history, indexing status, structured logs, and audit events where implemented.
Evaluation complexity	Success depends on retrieval relevance, citation accuracy, fallback behaviour, latency, and failure analysis rather than UI success alone.	Run the 31-question corpus during Sprint 4 and archive retrieved chunks, scores, citations, fallback flags, latency, and reviewer judgement.

Architecture Summary

EduRAG is organised as a modular monolith with four modules: DocumentProcessor, VectorRetriever, LLMResponseGeneration, and WebApplication. This keeps the MVP simple enough for a student team to implement and debug while preserving clean module boundaries for future expansion.

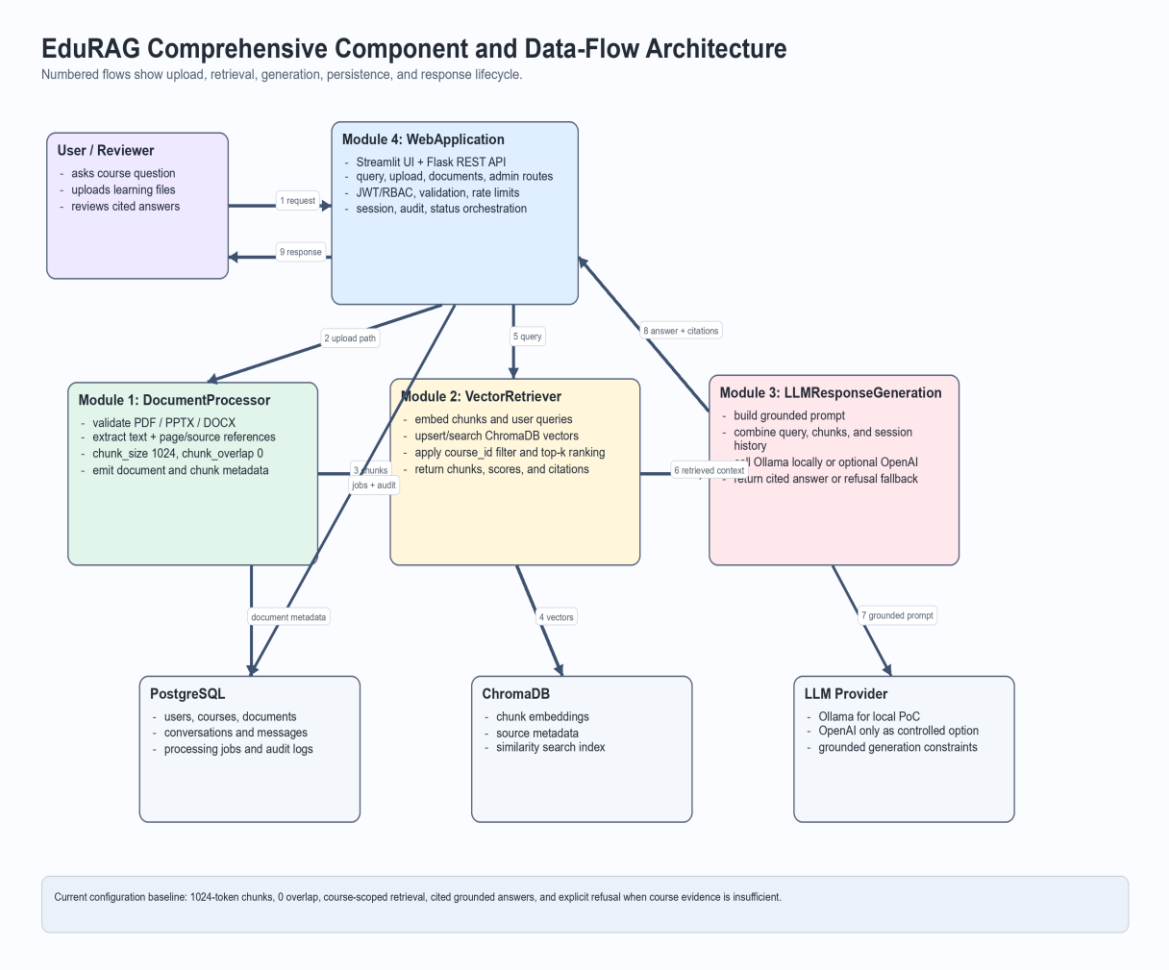


Figure 4. Comprehensive component and data-flow architecture.

The component design makes the system boundaries explicit. WebApplication is the only external entry point, while retrieval and generation remain internal modules. It also shows the complete data packages passed between modules: upload requests carry course and document metadata, query requests carry the original user question and session context, and the generation module receives both the user query and retrieved evidence before producing an answer.

Module Responsibilities

EduRAG is decomposed into four modules, each with a single responsibility. The full interface specifications are in Appendix B; the descriptions below convey the design intent at a level sufficient to understand the architecture as a whole.

DocumentProcessor

Responsible for the document ingest pipeline. When a Lecturer uploads a file, DocumentProcessor parses its text (supporting PDF, PPTX, and DOCX formats), applies the current implementation baseline of chunk_size=1024 with chunk_overlap=0, generates a

vector embedding for each chunk using the SentenceTransformers all-MiniLM-L6-v2 model (Reimers and Gurevych, 2019), and writes the resulting embeddings to ChromaDB. Each record is tagged with a course_id metadata field so that subsequent retrieval can be scoped correctly to a single course.

VectorRetriever

Responsible for semantic search at query time. It receives the student's question, generates its embedding using the same model used during ingest, and performs a cosine similarity search against the ChromaDB collection filtered by the active course_id. It returns the top-k most relevant chunks together with their similarity scores, which the next module uses to assess grounding confidence.

LLMResponseGeneration

Responsible for answer generation. It receives the original question and the retrieved chunks, constructs a context-grounded prompt, and calls the configured language model to generate a response. The module attaches source citations derived from the chunk metadata. When no chunks exceed the similarity threshold, the module avoids presenting an unsupported course-grounded answer and returns a clear message that the answer cannot be verified from the available course materials. In the development and PoC demonstration environment this module calls Ollama by default; OpenAI remains only an optional controlled-demo provider if required.

WebApplication

The only externally exposed module, responsible for orchestrating the other three. In the current design it acts as a lightweight Flask-based application service layer sitting behind the Streamlit prototype interface, enforces JWT-based authentication (Jones et al., 2015), and applies role-based access control across three roles: Student (query only), Lecturer (upload and content management), and Admin (system configuration and user management). It also enforces per-user rate limits, manages conversation sessions stored in PostgreSQL JSONB columns, and writes all significant events to the audit log.

Because EduRAG is an LLM-enabled system, the security design also treats prompts, retrieved chunks, and uploaded files as untrusted input. Prompt templates therefore keep course-scoping and citation rules in the highest-priority system instructions, uploaded documents are validated for type and size before parsing, and suspicious model outputs can be blocked, relabelled, or logged for follow-up instead of being shown blindly to end users. This directly addresses prompt-injection and improper-output-handling risks highlighted in the OWASP guidance for LLM applications (OWASP Foundation, 2025).

RAG Pipeline

The core behaviour of EduRAG can be understood as two sequential phases that together constitute the retrieval-augmented generation pipeline (Lewis et al., 2020).

Phase 1 — Document Ingest

1. A Lecturer uploads a course document (PDF, PPTX, or DOCX) through the WebApplication module.
2. DocumentProcessor extracts raw text from the file using format-specific parsers.
3. The extracted text is split using the current accepted RAG baseline of chunk_size=1024 and chunk_overlap=0. This aligns with the Sprint 3 benchmarking plan's tested 1024 option and the corrected local repo/report baseline used for Assessment 2.

PPTX ingestion needs slightly different handling from PDF or DOCX because slide decks are usually composed of short bullet fragments rather than continuous prose. For that reason, the parsing step preserves slide boundaries, titles, and local bullet structure before chunk assembly so each chunk keeps enough presentation context for later citation. Chunk assembly should still use the accepted retrieval baseline of chunk_size=1024 and chunk_overlap=0; smaller slide-derived chunks may be retained only when combining slides would weaken citation traceability.

4. SentenceTransformers all-MiniLM-L6-v2 encodes each chunk as a 384-dimensional dense vector embedding.
5. Each embedding is written to ChromaDB together with metadata that includes the course_id, document name, and chunk index.
6. The system confirms the upload and indexing status to the Lecturer.

Phase 2 — Query Flow

1. A Student submits a natural language question and selects the relevant course context.
2. WebApplication validates the JWT, checks the rate limit, and forwards the request.
3. VectorRetriever embeds the question using the same encoding model and searches ChromaDB with a cosine similarity query scoped to the active course_id.
4. The top-k most relevant chunks (default k=5) are returned with their similarity scores.
5. LLMResponseGeneration builds a context-grounded prompt from the retrieved chunks and the original question.
6. The prompt is sent to the configured language model: Ollama for local development and PoC demonstration, with OpenAI available only as a configurable controlled-demo option if required.

7. The generated answer is returned to the Student with source citations. If no chunks exceed the similarity threshold, the response states that the answer cannot be verified from the available course materials and avoids unsupported citations.
8. The interaction is stored in the session record and the audit log.

The DFD Level 1 in Appendix G provides a complementary view of the same flows at context and internal decomposition levels. The upload exception/recovery sequence diagram in that appendix traces the individual service calls and data movements in chronological order.

The current retrieval thresholds should be read as initial prototype baselines rather than permanent production guarantees. Precision@5 and Recall@10 were selected because they are easy to interpret in supervisor review, align with common top-k retrieval evaluation practice, and can be measured repeatedly on a curated course-specific query set with manually labelled relevant chunks (Thakur et al., 2021). To complement those labelled IR metrics, the team also plans to use a lightweight RAG-specific framework such as Ragas to monitor answer faithfulness and context relevance while chunking, prompts, and thresholds are being tuned (Es et al., 2023).

RAG Evaluation Strategy

The RAG evaluation strategy is intended to validate whether Paedia answers student questions with relevant course evidence, accurate citations, and appropriate refusal/fallback behaviour when the knowledge base does not contain enough support. It is designed to complement functional testing because a RAG system can pass interface tests while still retrieving weak chunks or producing unsupported answers.

Retrieval edge-case taxonomy. The Sprint 3 evidence defines four retrieval edge cases that Sprint 4 validation should keep separate: no chunks above threshold, borderline similarity matches, nonsense or accidental messages, and coherent off-topic questions. Separating these cases improves the evaluation because a no-match course question, a low-confidence partial match, and an unrelated general question should not all receive the same system response.

Evaluation baseline. Sprint 4 validation should use the accepted retrieval baseline: chunk_size=1024, chunk_overlap=0, top_k=5, similarity distance threshold=0.65, all-MiniLM-L6-v2 embeddings with 384 dimensions, and ChromaDB metadata filtering by course_id. The 1024/0 chunking configuration is treated as the Assessment 2 standard because it is the current repo/report baseline, even though one exported meeting note contains an inconsistent 2,024-token reference.

Evaluation dataset. The current labelled corpus should be tests/test_queries.json, which contains 31 test questions across easy factual, hard reasoning, off-topic, and ambiguous categories. This is a strong starting point for Sprint 4 because it tests both normal course questions and cases where Paedia should avoid over-answering. The dataset should still be

expanded before production use to include lecturer-approved expected citations, expected evidence pages or chunks, more subjects, and larger documents.

Evaluation area	What to test	Metric / evidence to collect	Acceptance direction
Groundedness	Whether generated answers are supported by retrieved chunks.	Manual judgement on the labelled corpus, supported by Ragas only if added and executed.	Answers should be based on retrieved course material, not unsupported model knowledge.
Citation accuracy	Whether each cited source corresponds to the retrieved evidence used in the answer.	Citation/source match rate and reviewer notes for incorrect or missing citations.	Citations should point to the correct document or chunk metadata.
Retrieval relevance	Whether top-k retrieved chunks are useful for the question.	Precision@5, Recall@10 where expected evidence is labelled, and qualitative review for hard reasoning cases.	Relevant chunks should appear in the top results often enough to support answer generation.
Fallback correctness	Whether no-match, borderline, nonsense, and off-topic cases trigger distinct safe behaviours instead of hallucinated answers.	Pass/fail count across off-topic, insufficient-evidence, accidental/nonsense, and borderline-similarity questions.	Unsupported questions should produce the configured fallback/refusal response; borderline matches should include a confidence caveat and verifiable sources.
Latency	Whether the query pipeline remains usable for student workflows.	End-to-end response time from API request to generated answer.	Prototype target should be responsive enough for iterative study use; slow cases should be logged.
Course isolation	Whether questions for one course can retrieve content from another course.	Cross-course negative tests using course_id metadata filters.	No retrieved evidence should come from unauthorised or unrelated course collections.

Table 4. RAG evaluation strategy for Sprint 4 validation.

Evidence status. This strategy should be reported as the planned Sprint 4 validation approach unless the team has executed the tests and archived the results. Final evidence should include the test corpus version, configuration values, retrieved chunks, similarity scores, generated answer, citations, fallback flag, latency, reviewer judgement, per-category results, failed-query examples, and any tuning decisions made after evaluation.

Technical Risk and Responsible Use Controls

The A2 risk view focuses on design risks that could weaken the prototype or reduce trust in its educational use. The mitigations are expressed as system controls and validation checks rather than named-person ownership, because Assessment 2 is primarily evaluating the design baseline.

Risk area	Potential impact	Mitigation or validation control
RAG quality	Weak retrieval or unsupported generation could produce misleading	Use thresholded retrieval, source citations, refusal/fallback labelling

	study answers.	when evidence is insufficient, and planned Precision@5/Recall@10, citation accuracy, and groundedness checks.
Privacy and data scope	Uploaded materials, conversations, and logs may contain sensitive study or account data.	Collect only data needed for authentication, course-scoped retrieval, document management, session continuity, and prototype accountability.
Upload and indexing failure	Large files, unsupported formats, or parsing errors could block the knowledge base.	Expose indexing status, failed states, retry/error information, and format-specific parser checks.
Latency	Slow query or indexing performance could make iterative study use frustrating.	Measure API-to-answer latency, indexing throughput, and slow cases during Sprint 4 validation.
RBAC and course leakage	Incorrect access control could expose admin features or another course's material.	Validate JWT/RBAC enforcement, role navigation, course_id filtering, and cross-course negative tests.
PDF/export and submission readiness	Generated PDFs or final report exports could lose formatting or citation clarity.	Use final PDF review, visual render checks, and two-person submission review before upload.

Deployment Topology

EduRAG is deployed as a Docker Compose multi-container stack. Three containers run on the same host during development and sprint demonstration: the Flask application container (WebApplication + all Python modules), the ChromaDB container (vector store, accessed via its HTTP API), and the PostgreSQL container (relational state, sessions, and audit log). Requests from the student browser reach the Flask container on a single exposed port; all inter-service communication stays on an internal Docker network.

The LLM provider is the only external dependency. In development, Ollama runs as a locally installed service on the host machine and is accessed from within the Docker container over the Docker host network bridge. If a controlled demonstration requires a hosted model, the same module can switch to the OpenAI API endpoint by changing a single environment variable, which is how the hybrid LLM strategy is implemented without duplicating application code.

This topology keeps the stack portable across developer machines and reproducible for the supervisor demonstration. The full deployment specification, including service definitions and

environment variable contracts, is documented in the project repository's docker-compose.yml.

The data model separates relational state from vector state. PostgreSQL stores users, roles, sessions, and audit-oriented records, while ChromaDB stores embeddings and metadata used for semantic retrieval. This separation is important because it supports both the system's security model and its retrieval workflow.

Storage sizing. The Sprint 3 storage estimate models a realistic prototype corpus of 10 courses, 50 documents per course, and 500 pages per document. With an assumed three chunks per page, this produces approximately 750,000 chunks. The estimate allocates about 2.5-3 GB for ChromaDB embeddings and metadata, 1-2 GB for PostgreSQL metadata, sessions, jobs, and audit state, and about 5 GB for raw uploaded documents. The resulting total of approximately 8.5-10 GB is manageable for the Docker-based prototype while still large enough to justify separating relational state from vector storage.

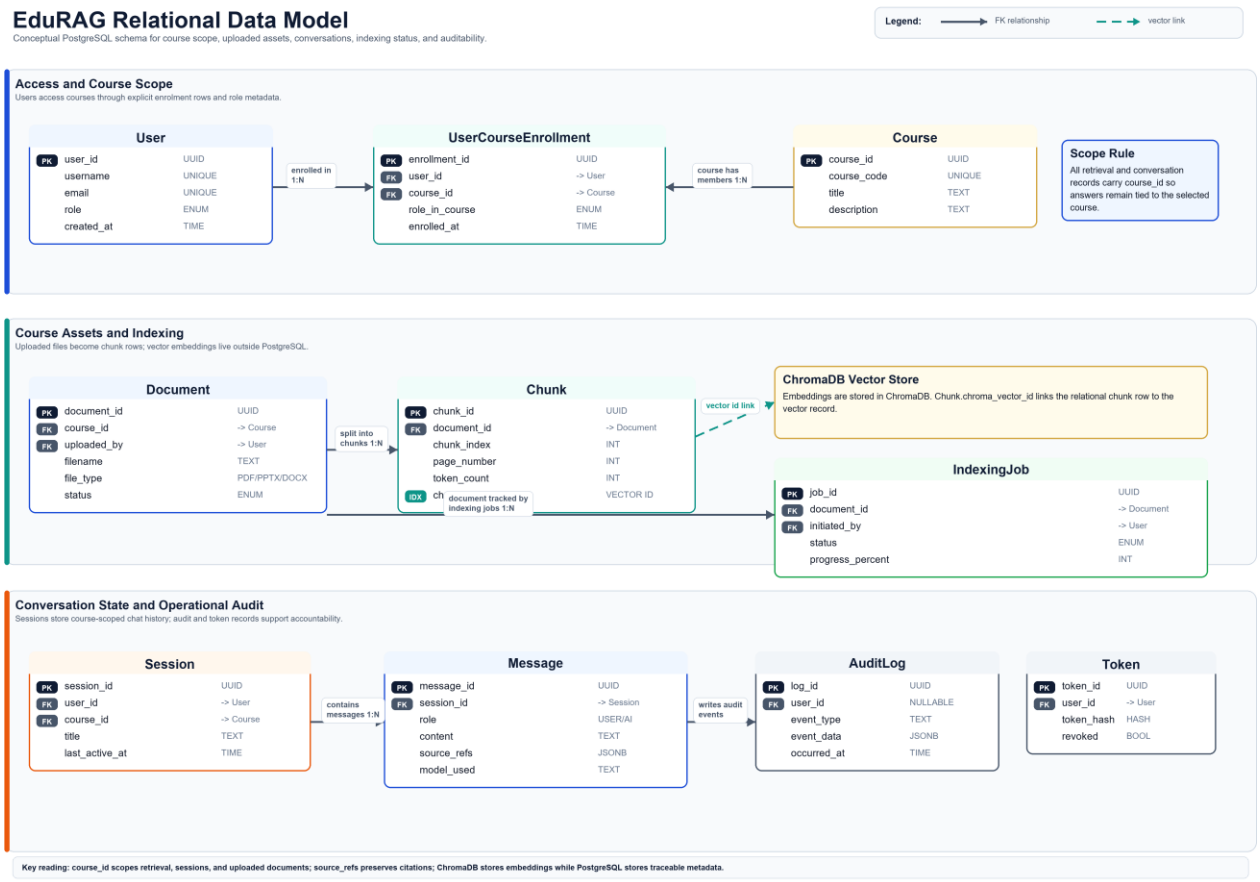


Figure 5. Conceptual data model for relational state, sessions, and course assets.

Appendix G contains the entity relationship diagram, DFD Level 1 internal process decomposition, and upload exception/recovery sequence diagram for the complete system.

Appendix H adds the initial interface wireframes that show how the same requirements appear in Student, Lecturer, and Admin workflows.

Conclusion

The Sprint 2 design work, strengthened by Sprint 3 consultation and prototype evidence, establishes a coherent and implementable foundation for EduRAG. The system has a well-defined target context (course-specific student support), a clear architectural approach (RAG pipeline built on ChromaDB, SentenceTransformers, and a configurable LLM provider), and a requirements baseline that distinguishes what must be delivered from what is desirable and what is explicitly out of scope. Taken together, these artefacts give the implementation sprints a concrete starting point rather than an open-ended problem statement.

The requirements specification demonstrates that the project is appropriately difficult for a CSIT321 capstone. The must-have requirements alone span document ingestion, vector retrieval, grounded answer generation, role-based access control, and audit logging. Each of these involves genuine engineering decisions, particularly around chunking strategy, embedding model selection, similarity threshold tuning, and the integration of two LLM providers (Ollama for local development and PoC demonstration, with OpenAI retained only as an optional controlled-demo provider). The unsupported-answer handling mechanism adds an additional layer of epistemic honesty that distinguishes EduRAG from a generic chatbot wrapper.

The most important outcome of this Assessment 2 report is not simply that tools have been selected, but that those selections are now tied to a requirement-driven design. Every core technology in the stack is referenced in at least one functional or non-functional requirement, and the traceability matrix links each major user-facing requirement back to one or more user stories. This traceability is what makes the design reviewable and the implementation verifiable. The next practical step is further supervisor confirmation of the proposed scope, difficulty level, and Sprint 4 validation evidence, after which the team can move into implementation sprints with a shared understanding of what constitutes a complete and correct system.

References

Chroma (2026) Chroma documentation. Available at: https://docs.trychroma.com/ (Accessed: 14 April 2026).

Docker Inc. (2026) Docker Compose documentation. Available at: https://docs.docker.com/compose/ (Accessed: 14 April 2026).

LangChain (2026) LangChain documentation. Available at: https://python.langchain.com/docs/introduction/ (Accessed: 14 April 2026).

LlamaIndex (2026) LlamaIndex documentation. Available at: https://docs.llamaindex.ai/ (Accessed: 14 April 2026).

Ollama (2026) Ollama documentation. Available at: https://docs.ollama.com/ (Accessed: 14 April 2026).

OpenAI (2026) OpenAI API documentation. Available at: https://platform.openai.com/docs/introduction (Accessed: 14 April 2026).

PostgreSQL Global Development Group (2026) PostgreSQL documentation. Available at: https://www.postgresql.org/docs/ (Accessed: 14 April 2026).

Sentence Transformers (2026) Sentence Transformers documentation. Available at: https://sbert.net/ (Accessed: 14 April 2026).

Streamlit (2026) Streamlit documentation. Available at: https://docs.streamlit.io/ (Accessed: 14 April 2026).

Flask (2026) Flask documentation. Available at: https://flask.palletsprojects.com/ (Accessed: 14 April 2026).

Jones, M., Bradley, J. and Sakimura, N. (2015) RFC 7519: JSON Web Token (JWT). Internet Engineering Task Force (IETF). Available at: https://datatracker.ietf.org/doc/html/rfc7519 (Accessed: 14 April 2026).

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S. and Kiela, D. (2020) 'Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks', Advances in Neural Information Processing Systems, 33, pp. 9459–9474. Available at: https://arxiv.org/abs/2005.11401 (Accessed: 14 April 2026).

Reimers, N. and Gurevych, I. (2019) 'Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks', Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP), pp. 3982–3992. Available at: https://arxiv.org/abs/1908.10084 (Accessed: 14 April 2026).

Schwaber, K. and Sutherland, J. (2020) The Scrum Guide: The Definitive Guide to Scrum: The Rules of the Game. Scrum.org. Available at: https://scrumguides.org/scrum-guide.html (Accessed: 14 April 2026).

Siriwardhana, S., Weerasekera, R., Wen, E., Kaluarachchi, T., Rana, R. and Nanayakkara, S. (2023) 'Improving the Domain Adaptation of Retrieval Augmented Generation (RAG) Models for Open Domain Question Answering', Transactions of the Association for Computational Linguistics, 11, pp. 1–17. Available at: https://arxiv.org/abs/2210.02627 (Accessed: 14 April 2026).

Thakur, N., Reimers, N., Rueckle, A., Srivastava, A. and Gurevych, I. (2021) 'BEIR: A Heterogenous Benchmark for Zero-shot Evaluation of Information Retrieval Models'. Available at: https://arxiv.org/abs/2104.08663 (Accessed: 20 April 2026).

Es, S., James, J., Espinosa-Anke, L. and Schockaert, S. (2023) 'Ragas: Automated Evaluation of Retrieval Augmented Generation'. Available at: https://arxiv.org/abs/2309.15217 (Accessed: 20 April 2026).

OWASP Foundation (2025) 'LLM01:2025 Prompt Injection'. Available at: https://genai.owasp.org/llmrisk/llm01-prompt-injection/ (Accessed: 20 April 2026).

Australian Government Department of Education (2026) Key findings from the 2024 Higher Education Student Statistics. Available at: https://www.education.gov.au/higher-education-statistics/student-data/selected-higher-education-statistics-2024-student-data/key-findings-2024-higher-education-student-statistics (Accessed: 1 May 2026).

Chegg (2025) Chegg Global Student Survey 2025: 80% of undergraduates worldwide have used GenAI to support their studies - but accuracy a top concern. Available at: https://www.chegg.com/about/newsroom/press-release/chegg-global-student-survey-2025-80-of-undergraduates-worldwide-have-used-genai-to-support-their-studies-but-accuracy-a-top-concern (Accessed: 1 May 2026).

EDUCAUSE (2024) 2024 EDUCAUSE Action Plan: AI Policies and Guidelines. Available at: https://library.educause.edu/resources/2024/5/2024-educause-action-plan-ai-policies-and-guidelines (Accessed: 1 May 2026).

Freeman, J. (2025) Student Generative AI Survey 2025. HEPI Policy Note 61. Higher Education Policy Institute. Available at: https://www.hepi.ac.uk/reports/student-generative-ai-survey-2025/ (Accessed: 1 May 2026).

Grand View Research (2024) AI in Education Market Size to Reach $32.27 Billion by 2030. Available at: https://www.grandviewresearch.com/press-release/global-artificial-intelligence-ai-education-market (Accessed: 1 May 2026).

UNESCO (2023) Guidance for Generative AI in Education and Research. Available at: https://www.unesco.org/en/articles/guidance-generative-ai-education-and-research (Accessed: 1 May 2026).

University of Nebraska-Lincoln Center for Transformative Teaching (n.d.) Organize Content Well. Available at: https://teaching.unl.edu/resources/large-enrollment/organize-content-well/ (Accessed: 1 May 2026).

University of Wollongong (2025) Annual Report 2024. Available at: https://documents.uow.edu.au/content/groups/public/@web/@pmcd/@smc/documents/doc/uow279096.pdf (Accessed: 1 May 2026).

University of Wollongong (2026) New UOW Liverpool Campus expands university access in South Western Sydney. Available at: https://www.uow.edu.au/media/2026/new-uow-liverpool-campus-expands-university-access-in-south-western-sydney.php (Accessed: 1 May 2026).

Glossary

Term	Meaning in this project

RAG	Retrieval-Augmented Generation, where retrieved source content is provided to the language model before answer generation.
Embedding	A numeric vector representation of text used for similarity search.
Vector database	A storage system for embeddings that supports similarity-based retrieval.
ChromaDB	The selected vector store for course-scoped embedding persistence and retrieval.
JWT	JSON Web Token used for stateless authentication.
RBAC	Role-Based Access Control; used to separate Student, Lecturer, and Admin permissions.
MoSCoW	Prioritisation model dividing requirements into Must, Should, Could, and Won't have.
MVP	Minimum Viable Product; the smallest usable version of EduRAG that demonstrates core value.
Modular monolith	A single deployable application with explicitly separated internal modules.

Appendix A. Technology Decision Matrix

Component	Decision	Primary Rationale
Core web/API layer	Current Sprint 4 direction: Flask-based WebApplication module with documented REST-style endpoints for authentication, query, upload, documents, sessions, indexing status, and admin monitoring.	Keeps the MVP in one Python stack while still exposing clear module boundaries; Sprint 3 evidence validates the Python app-service direction, while Sprint 4 tasking names Flask as the API implementation path.
Frontend (MVP)	Streamlit prototype with role-based views for Student, Lecturer, and Admin.	Fastest path to a demonstrable interface; confirmed as appropriate for the PoC and Assessment 2 design stage.
Frontend (stretch)	React frontend after the core Streamlit prototype is stable.	Provides a production-grade migration path without risking the MVP schedule.
LLM provider	Ollama for local development and PoC demonstration; OpenAI API as an optional controlled-demo provider if required.	Aligns with supervisor-validated local inference while retaining a configurable hosted-model fallback for demonstration quality or availability.
Vector database	ChromaDB with course_id metadata filtering and persistence.	Supports scoped retrieval, citation traceability, and simple local deployment.
Embedding model	SentenceTransformers all-MiniLM-L6-v2, 384-dimensional embeddings.	Lightweight, fast, well-supported, and suitable for prototype-scale semantic retrieval.
Relational database	PostgreSQL for users, roles, courses, documents, sessions, messages, indexing jobs, and audit events.	Provides reliable persistence, relational integrity, JSONB flexibility, and RBAC-friendly data modelling.
Authentication	JWT-based authentication with role-based access control for Student, Lecturer, and Admin roles.	Separates user capabilities and protects administrative/course-management functions.
Deployment	Docker Compose with app, PostgreSQL, ChromaDB, and optional LLM service containers.	Improves portability across team machines and provides a clear demo/development environment.
Architecture	Modular monolith with DocumentProcessor, VectorRetriever, LLMResponseGeneration, and WebApplication modules.	Keeps integration simple while preserving clear technical ownership and module contracts.
Inter-module communication	Direct Python calls between internal modules, surfaced externally through WebApplication endpoints.	Reduces microservice overhead while still allowing API-driven UI integration.
File storage	Local Docker volume for uploaded prototype documents.	Sufficient for assessment-stage evidence without introducing cloud storage complexity.
Caching	Lightweight in-memory caching for repeated configuration/model lookups where appropriate.	Improves responsiveness without adding another infrastructure dependency.
Indexing jobs	Asynchronous upload/indexing status model with pending, processing, indexed, and failed states.	Prevents large documents from blocking the user interface and supports clear error recovery.
Logging and audit	Structured logs plus admin-facing	Supports debugging, accountability, and

	audit/metrics views where implemented.	responsible educational-AI monitoring.
Error handling	Graceful user-facing errors with fail-fast internal validation for unsafe states.	Protects user trust while exposing developer-visible failures early.
Testing	Unit tests, integration checks, and RAG benchmark/evaluation corpus.	Matches the project risk profile: parsing, retrieval, citation, fallback, latency, and course isolation all need evidence.
Document formats	PDF, PPTX, and DOCX ingestion target.	Covers the most common course-material formats used by students and lecturers.
Course scoping	course_id filtering across retrieval, documents, sessions, and access control.	Prevents cross-course leakage and supports the supervisor-identified differentiation of course-level isolation.

Appendix B. Module Interface Specifications

Module 1 - DocumentProcessor

Interface Name	DocumentProcessor
Input	file_path (str): absolute path to the uploaded document (PDF, PPTX, or DOCX) file_type (str): MIME type, e.g. 'application/pdf' course_id (str): identifier of the course the document belongs to
Output	chunks (List[DocumentChunk]): list of text chunks, each containing: • chunk_id (str) • text (str) • source_file (str) • page_number (int) • course_id (str)
Methods	load_document(file_path, file_type) -> RawDocument extract_text(doc: RawDocument) -> str chunk_text(text: str, chunk_size=1024, overlap=0) -> List[str] process(file_path, file_type, course_id) -> List[DocumentChunk]
Dependencies	PyMuPDF (fitz) or pdfplumber for PDF extraction python-pptx for PPTX extraction python-docx for DOCX extraction LangChain RecursiveCharacterTextSplitter for chunking
Error Handling	Raises FileNotFoundError if file path is invalid Raises UnsupportedFileTypeError if file type is not PDF, PPTX, or DOCX Returns empty list and logs a warning if no extractable text is found
Called By	Module 4 (WebApplication) upon document upload event; triggers Module 2 downstream

Module 2 - VectorRetriever

Interface Name	VectorRetriever
Input (Indexing)	chunks (List[DocumentChunk]): output from Module 1 course_id (str): scope the embeddings to the relevant course knowledge base

Input (Retrieval)	query (str): the student's natural-language question course_id (str): scope the search to a specific course top_k (int, default=5): number of most relevant chunks to return
Output (Indexing)	IndexResult: object containing chunk_count (int), success (bool), errors (List[str])
Output (Retrieval)	retrieved_chunks (List[RetrievedChunk]): each chunk includes: • text (str) • source_file (str) • page_number (int) • similarity_score (float)
Methods	generate_embeddings(chunks: List[DocumentChunk]) -> List[EmbeddingVector] add_to_index(embeddings, chunks, course_id) -> IndexResult search(query: str, course_id: str, top_k: int) -> List[RetrievedChunk] delete_from_index(course_id: str, source_file: str) -> bool
Dependencies	SentenceTransformers (all-MiniLM-L6-v2) for embedding generation ChromaDB for vector storage and similarity search NumPy for vector operations
Error Handling	Raises IndexNotFoundError if course_id has no existing index and retrieval is attempted Returns empty list with warning log if no chunks exceed similarity threshold Raises EmbeddingGenerationError if the embedding model fails to produce vectors
Called By	Module 3 (LLM Response Generation) for retrieval Module 1 output triggers indexing after document upload

Module 3 - LLMResponseGeneration

Interface Name	**LLMResponseGeneration**
Input	query (str): the original student question retrieved_chunks (List[RetrievedChunk]): context from Module 2 conversation_history (List[Message], optional): prior turns for multi-turn support config (LLMConfig): model

	name, temperature, max_tokens
Output	response (GeneratedResponse): contains: • answer (str) • source_references (List[SourceRef]) • model_used (str) • token_count (int)
Methods	build_prompt(query, chunks, history) -> str generate(query, chunks, config, history) -> GeneratedResponse format_sources(chunks: List[RetrievedChunk]) -> List[SourceRef]
Dependencies	LangChain for prompt management and chain execution. Ollama or OpenAI API as configurable LLM providers. python-dotenv for secure API key management.
Error Handling	Raises APIConnectionError if the LLM provider is unreachable. Raises TokenLimitExceededError if prompt exceeds the model context window and truncates oldest history first. Returns a clear unsupported-answer message if no relevant chunks are retrieved; no fabricated citations are attached.
Called By	Module 4 (WebApplication) on each student query submission

Module 4 - WebApplication

Interface Name	**WebApplication**
Current interaction handlers / future API endpoints	POST /api/query — accepts { query, course_id }; returns GeneratedResponse POST /api/upload — accepts multipart file + course_id; triggers Module 1 DELETE /api/document/{doc_id} — removes document and its embeddings GET /api/metrics — returns system usage statistics (Admin only) GET /api/logs — returns audit log entries (Admin only) POST /api/auth/login — issues JWT on valid credentials
Input (Query Flow)	HTTP POST request body: { query: str, course_id: str, session_id: str (optional) }

Output (Query Flow)	HTTP 200 JSON response: { answer: str, sources: List[SourceRef], session_id: str }
Authentication	JWT-based authentication with Role-Based Access Control Roles: Student, Lecturer, Admin Tokens expire after 8 hours; refresh tokens stored in PostgreSQL for revocation support
Dependencies	Flask for the current application service layer, with Streamlit as the MVP UI client. Module 1, Module 2, and Module 3 via direct Python imports. PostgreSQL for session management, user data, and audit logging.
Error Handling	Returns HTTP 400 for malformed requests. Returns HTTP 401 for unauthenticated requests. Returns HTTP 403 for authenticated requests lacking the required role. Returns HTTP 429 when rate limits are exceeded. Returns HTTP 500 with a safe error message for internal pipeline failures.
Calls	Module 1 on document upload Module 2 for retrieval Module 3 for response generation

Inter-module communication summary

From Module	To Module	Trigger	Data Passed
Module 4 WebApplication	Module 1 DocumentProcessor	User uploads a document	file_path, file_type, course_id
Module 1 DocumentProcessor	Module 2 VectorRetriever	Document processing complete	List[DocumentChunk], course_id
Module 4 WebApplication	Module 2 VectorRetriever	Student submits a query	query, course_id, top_k
Module 2 VectorRetriever	Module 3 LLM Generation	Retrieval complete	List[RetrievedChunk], query
Module 3 LLM Generation	Module 4 WebApplication	Response generated	GeneratedResponse (answer + sources)

Appendix C. Full Functional Requirements

ID	Name	Description	Priority	Module	Acceptance Criterion
FR-001	User Authentication	The system shall authenticate users via username and password, issuing a JWT upon successful login.	Must	M4	Valid credentials receive a signed JWT; invalid credentials receive HTTP 401.
FR-002	Role-Based Access Control	The system shall enforce three distinct roles (Student, Lecturer, Admin) with different permissions per endpoint.	Must	M4	Role enforcement verified by unit tests on all protected endpoints.
FR-003	Document Upload (PDF)	Lecturers shall be able to upload PDF course materials through the web interface.	Must	M1, M4	A valid PDF of up to 100 MB uploads successfully and returns a job ID.
FR-004	Text Extraction	The system shall extract text content from uploaded PDF documents.	Must	M1	Extracted text is non-empty for a non-scanned PDF; warnings logged for scanned PDFs.
FR-005	Document Chunking	The system shall split extracted text	Must	M1	Default processing uses

		using the current RAG baseline of chunk_size=1024 and chunk_overlap=0.			chunk_size=1024 and chunk_overlap=0, with chunk metadata retained for source citation and retrieval.
FR-006	Embedding Generation	The system shall generate 384-dimensional vector embeddings for each chunk using all-MiniLM-L6-v2.	Must	M2	Every chunk has a non-null embedding vector of length 384.
FR-007	Vector Indexing	The system shall store embeddings in ChromaDB indexed by course_id metadata.	Must	M2	Stored chunks retrievable by course_id filter; count matches chunks generated.
FR-008	Semantic Search	The system shall retrieve the top-K most similar chunks for a given query, scoped to a specific course.	Must	M2	search() returns a list of length top_k with non-decreasing similarity scores.
FR-009	Natural Language Query	Students shall be able to submit natural language questions through the web interface.	Must	M4	Query input accepts strings up to 1000 characters and returns HTTP 200 on valid input.
FR-010	LLM-Grounded	The system shall	Must	M3	Answers contain

	Answers	generate answers using an LLM, conditioned on the retrieved chunks.			information present in retrieved chunks on the curated test set.
FR-011	Source Citation Display	Every generated answer shall include references to the source document(s) and page number(s).	Must	M3, M4	≥ 95% of answers include at least one valid source reference.
FR-012	Async Indexing with Status	Document indexing shall run asynchronously; clients shall poll a status endpoint for progress.	Must	M1, M4	Upload returns HTTP 202 with job_id; status endpoint reflects progress and final state.
FR-013	Multi-Course Support	The system shall maintain separate, isolated knowledge bases per course_id.	Must	M2	A query against course A does not return chunks from course B.
FR-014	Single-Turn Q&A	The system shall answer individual questions without requiring prior context.	Must	M3, M4	Standalone queries return sensible answers without conversation history.
FR-015	Structured Logging	All significant events (login, upload, query, error) shall be logged in structured	Must	M4	Log entries parseable as JSON; include timestamp, level, event type, user_id.

		JSON format.			
FR-016	Graceful Fallback — No Course Material Found	When no chunks exceed the similarity threshold, the system shall avoid presenting an unsupported course-grounded answer and return a clear message that the answer cannot be verified from the available course materials.	Must	M2, M3	Queries with no relevant course content return an unsupported-answer message; no fabricated citations or raw error is exposed to the user.
FR-017	Secrets Management	API keys and secrets shall be loaded from environment variables, never hardcoded or committed to git.	Must	All	Static analysis confirms no hardcoded secrets; .env file is in .gitignore.
FR-018	Password Hashing	User passwords shall be hashed with bcrypt before storage in PostgreSQL.	Must	M4	Database inspection shows no plaintext passwords; bcrypt cost factor ≥ 12.
FR-019	Document Upload (PPTX)	The system shall also accept PPTX files as course material input.	Should	M1	A valid PPTX of up to 100 MB uploads and indexes successfully.
FR-020	Document Upload	The system shall	Should	M1	A valid DOCX of up

	(DOCX)	also accept DOCX files as course material input.			to 100 MB uploads and indexes successfully.
FR-021	Multi-Turn Conversations	The system shall support follow-up questions that reference prior turns in the same session.	Should	M3, M4	Conversations with up to 5 prior turns produce contextually appropriate answers.
FR-022	Saved Conversation Sessions	Users shall be able to save and revisit prior conversation sessions.	Should	M4	Saved sessions persist across logins and display in chronological order.
FR-023	Admin Usage Dashboard	Admins shall see a dashboard displaying total queries, average response time, and active users.	Should	M4	Dashboard loads in under 3 seconds and displays metrics for the last 30 days.
FR-024	Audit Log Review	Admins shall be able to view an append-only audit log of significant events.	Should	M4	Audit log queryable by user, date range, and event type; log entries immutable.
FR-025	Rate Limiting	The system shall limit the number of queries per user per minute to prevent abuse.	Should	M4	Exceeding the limit returns HTTP 429; limit configurable per role.

FR-026	Document Management	Lecturers shall be able to view, list, and delete their uploaded documents.	Should	M4	Delete operation removes the document file, chunks, and embeddings atomically.
FR-027	Confidence Scoring	The system shall display a confidence indicator with each answer based on similarity scores.	Should	M2, M3, M4	Confidence shown as Low / Medium / High derived from mean top-K similarity.
FR-028	Retrieval Accuracy Benchmarks	The system shall include an automated test suite measuring Precision@5 and Recall@10.	Should	M2	Benchmark script runs in CI against a curated labelled query set; results archived per build.
FR-029	Export Conversation to PDF	Users shall be able to export a conversation session as a PDF document.	Could	M4	Exported PDF contains all Q&A pairs with source citations preserved.
FR-030	Topic Summarisation	Users shall be able to request a topic summary by specifying a document or chapter.	Could	M3	Summaries are under 500 words and reference the source material accurately.
FR-031	Query Suggestions	The system shall suggest follow-up	Could	M3, M4	At least 3 relevant suggestions shown

		questions after each answer.			with each response.
FR-032	Dark Mode	The web interface shall support a user-selectable dark theme.	Could	M4	Toggle switches all UI elements to a dark color scheme; preference persisted.
FR-033	Usage Analytics Export	Admins shall be able to export usage metrics as CSV.	Could	M4	Export produces a valid CSV file with all visible dashboard metrics.
FR-034	Real-Time Lecture Transcription	The system will not transcribe live lecture audio into course materials.	Won't	—	Out of scope; may be considered in a future iteration.
FR-035	Multi-Language UI	The system will not offer a multi-language user interface for the MVP.	Won't	—	English-only UI; multilingual support documented as future work.
FR-036	LMS Integration	The system will not integrate with Moodle or other Learning Management Systems.	Won't	—	Out of scope for the prototype.
FR-037	Mobile Native App	The system will not be distributed as a native iOS or	Won't	—	Web interface only; mobile access via responsive web

		Android application.			design only.
FR-038	Automated Grading	The system will not grade student assignments or assessments.	Won't	—	Educational tool only; grading is explicitly excluded.

Priority	**Count**	**Targeted Delivery**
Must	18	Sprints 5 — 8 (core MVP development)
Should	10	Sprints 8 — 10 (feature enrichment)
Could	5	Sprints 10 — 12 (stretch, if capacity permits)
Won't	5	Out of scope — not planned for this project

Appendix D. Full Non-Functional Requirements

ID	Category	Name	Target	Priority	Verification
NFR-001	Performance	Query Response Time	Under 5 seconds for the 95th percentile of queries against a corpus of typical size.	Must	Load test with 100 sample queries; measure p95 latency.
NFR-002	Performance	Document Upload Limit	Support upload and indexing of documents up to 100 MB or 500 pages.	Must	Upload a 100 MB / 500-page test PDF; confirm successful indexing.
NFR-003	Performance	Concurrent Users	Support 10 to 20 simultaneous users without significant degradation.	Should	Load test with 20 concurrent virtual users for 5 minutes.
NFR-004	Performance	Indexing Throughput	Process at least 50 pages per minute during background indexing.	Should	Benchmark indexing on a 100-page document; compute pages/min.
NFR-005	Quality	Retrieval Precision	Precision@5 >= 0.7 on the curated course-specific pilot query set.	Must	Run retrieval benchmark on a labelled pilot set; compute precision over top-5 results.
NFR-006	Quality	Retrieval Recall	Recall@10 >= 0.8 on the curated course-specific pilot query set.	Must	Run retrieval benchmark on a labelled pilot set; compute recall over top-10 results.

NFR-007	Quality	Citation Rate	At least 95% of generated answers shall include at least one source citation.	Must	Sample 100 answers; compute percentage with citations.
NFR-008	Quality	Hallucination Rate	Less than 5% of answers shall contain information not present in retrieved chunks.	Must	Manual review of 100 sampled answers against retrieved chunks.
NFR-009	Quality	User Satisfaction	Average user satisfaction score ≥ 4 out of 5 in User Acceptance Testing.	Should	UAT session with at least 10 participants; post-session survey.
NFR-010	Security	Authentication	All API endpoints except /api/auth/login shall require a valid JWT.	Must	Automated security scan; attempt unauthorized access to each endpoint.
NFR-011	Security	Password Storage	Passwords shall be hashed with bcrypt (cost factor ≥ 12) before storage.	Must	Code review; database inspection confirms no plaintext passwords.
NFR-012	Security	Transport Security	All client-server communication shall use HTTPS in production.	Must	Deployment configuration mandates TLS certificates.
NFR-013	Security	Input Validation	All user input shall be validated for type, length, and format before	Must	Unit tests cover invalid input cases for all endpoints.

			processing.		
NFR-014	Security	SQL Injection Prevention	All database queries shall use parameterised statements.	Must	Code review; static analysis with bandit or similar tool.
NFR-015	Security	Rate Limiting	API endpoints shall enforce per-user rate limits to prevent abuse.	Must	Load test exceeds limit; confirms HTTP 429 response.
NFR-016	Security	Audit Logging	All authentication events and data modifications shall be logged to an append-only audit log.	Must	Inspect audit log after performing test operations.
NFR-017	Security	Secrets Management	Secrets shall be loaded from environment variables; .env files shall be excluded from version control.	Must	Static scan confirms no hardcoded secrets; .gitignore verified.
NFR-018	Privacy	Minimal Data Collection	The system shall collect only username, email, role, hashed password, last login, and conversation history.	Must	Database schema review; confirm no extraneous columns.
NFR-019	Privacy	Access Control by Role	Users shall only be able to access their own conversation history.	Must	Unit tests verify user A cannot access user B's sessions.
NFR-020	Usability	Browser	The web interface shall	Should	Manual cross-browser test

		Support	function correctly on the latest versions of Chrome, Firefox, and Safari.		on all three browsers.
NFR-021	Usability	Basic Accessibility (MVP)	The MVP shall support keyboard navigation, sufficient colour contrast, and alt text on all images.	Must	Manual accessibility audit against a basic checklist.
NFR-022	Usability	Full Accessibility (Stretch)	The stretch React version shall comply with WCAG 2.1 AA.	Could	Automated WCAG scan with axe-core or similar tool.
NFR-023	Usability	Response Feedback	The UI shall display loading indicators during any operation taking longer than 1 second.	Should	Manual UI review; confirm loading states on all long operations.
NFR-024	Reliability	Graceful LLM Failure	If the LLM provider is unreachable, the system shall return retrieved chunks with an explanatory notice.	Must	Simulate LLM outage; confirm fallback response is returned.
NFR-025	Reliability	System Availability	The system shall make a best effort to be available during scheduled demo periods.	Should	Manual verification; no formal uptime SLA.

NFR-026	Reliability	Graceful Restart	A container restart shall not lose user session data or uploaded documents.	Must	Restart test with active sessions; confirm data persistence.
NFR-027	Maintainability	Module Boundaries	Each of the four modules shall be a self-contained Python package with a documented public interface.	Must	Code review; confirm __init__.py exports only the documented interface.
NFR-028	Maintainability	Test Coverage	Unit test coverage shall be at least 70% for core business logic modules.	Should	Run coverage.py; generate report.
NFR-029	Maintainability	Configuration Externalisation	All environment-specific configuration shall be externalised to environment variables or config files.	Must	Code review; no hardcoded URLs, paths, or credentials.
NFR-030	Portability	Dockerised Deployment	The entire system shall deploy via a single docker-compose up command on any Docker-capable host.	Must	Fresh install test on at least two different machines.

Appendix E. User Stories and Use Case Descriptions/Diagram

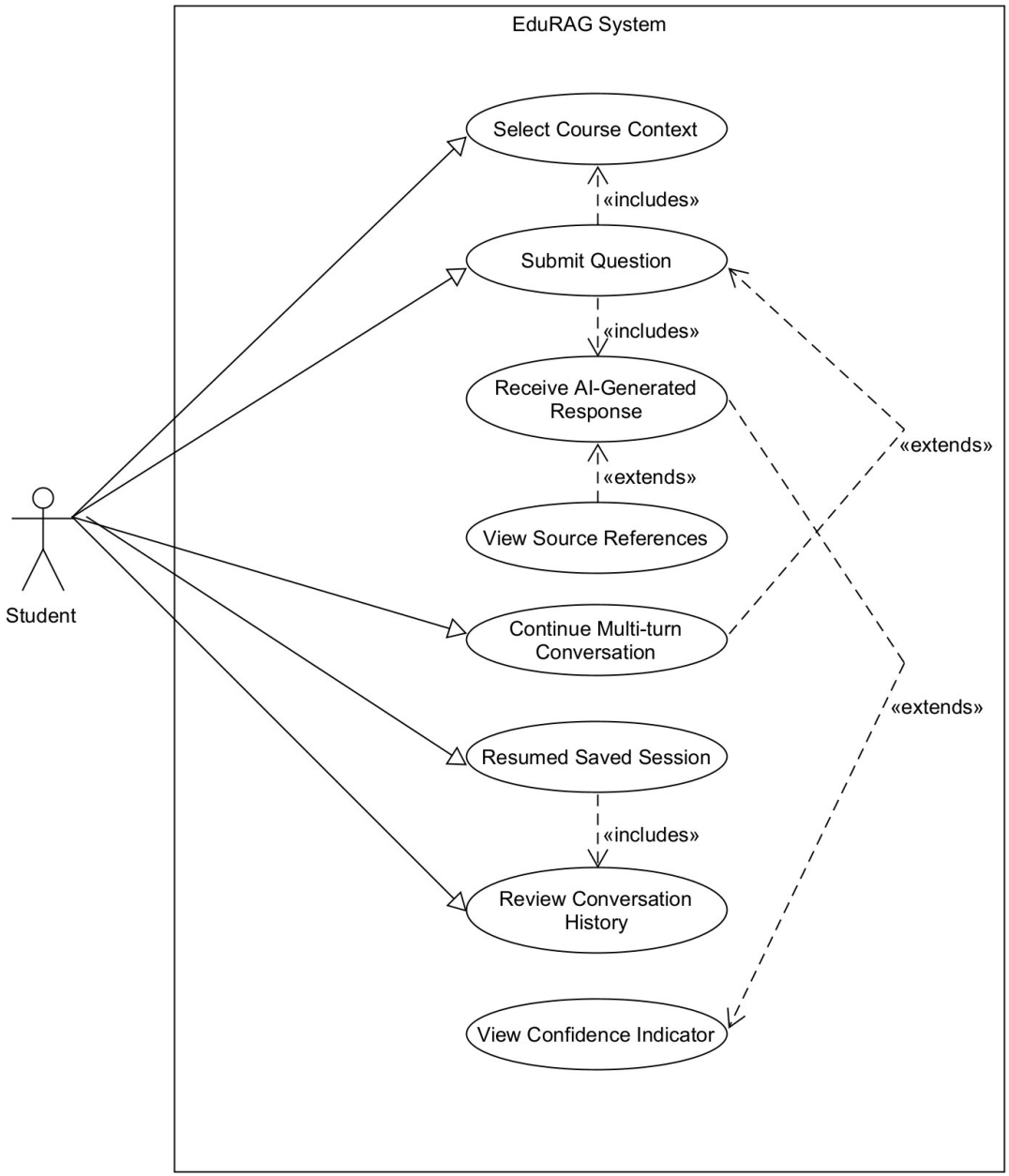
Role	ID	User story
Student	US-S1	As a Student, I want to ask natural language questions about my course materials so that I can find answers without remembering exact keywords.
Student	US-S2	As a Student, I want to see source citations with each answer so that I can verify where the information came from.
Student	US-S3	As a Student, I want to ask follow-up questions in the same conversation so that I can explore a topic in more depth without repeating context.
Student	US-S4	As a Student, I want to save and revisit previous conversations so that I can continue studying from where I left off.
Student	US-S5	As a Student, I want to select the relevant course before asking a question so that the system searches the correct knowledge base.

Student	US-S6	As a Student, I want to see a confidence indicator with each answer so that I know when I should double-check the response more carefully.
Student	US-S7	As a Student, I want the system to tell me clearly when no relevant material is found so that I do not rely on a made-up answer.
Student	US-S8	As a Student, I want to review my past questions and answers so that I can use the assistant as part of my revision process.
Lecturer	US-L1	As a Lecturer, I want to upload course materials (PDFs, slides) so that students can query them through the assistant.
Lecturer	US-L2	As a Lecturer, I want to organise uploaded materials by subject or topic so that the assistant retrieves contextually relevant content.
Lecturer	US-L3	As a Lecturer, I want to delete outdated materials from the knowledge base so that the assistant does not produce answers based on stale

		content.
Lecturer	US-L4	As a Lecturer, I want to preview how my documents are chunked and indexed so that I can verify the quality of text extraction.
Lecturer	US-L5	As a Lecturer, I want to view usage analytics for my course materials so that I can see which documents students reference most and identify knowledge gaps.
Lecturer	US-L6	As a Lecturer, I want to flag inaccurate responses generated by the assistant so that the development team can investigate and improve retrieval.
Admin	US-A1	As an Admin, I want to create and manage user accounts so that only authorised users can access the system.
Admin	US-A2	As an Admin, I want to assign Lecturers to courses so that uploaded materials remain scoped correctly.
Admin	US-A3	As an Admin, I want to reset passwords so that I can help locked-out users regain

		access.
Admin	US-A4	As an Admin, I want to configure LLM settings so that the system can use the selected response provider.
Admin	US-A5	As an Admin, I want to monitor usage metrics so that I can detect performance issues early.
Admin	US-A6	As an Admin, I want to review and export audit logs so that system changes remain traceable.
Admin	US-A7	As an Admin, I want to trigger re-indexing after bulk content changes so that embeddings stay current.

Student Use Cases


EduRAG System
Select Course Context
«includes»
Submit Question
«includes»
Receive AI-Generated Response
«extends»
«extends»
View Source References
Continue Multi-turn Conversation
«extends»
Resumed Saved Session
«includes»
Review Conversation History
View Confidence Indicator
Student

UC-01 Select Course Context

Description

A Student selects the course that their question belongs to so that the system searches the correct course knowledge base.

Actor

Student (Primary)

Trigger

Student opens the Q&A interface and chooses a course before asking a question.

Preconditions

Student is authenticated. Student has access to one or more courses.

Postconditions

The selected course becomes the active query context for the Student's current interaction.

Normal Flow

1. Student navigates to the Q&A interface.
2. System displays the list of available courses.
3. Student selects a course from the list.
4. System sets the selected course as the active context.
5. Student proceeds to ask a question within that course scope.

Alternative Flows

1a. If the Student does not select a course, system prompts the Student to choose one before submitting a question.

2a. If no courses are available, system displays a message that no accessible course context exists.

UC-02 Submit Question

Description

A Student submits a natural language question about the selected course and requests an answer from the system.

Actor

Student (Primary)

Trigger

Student enters a question in the Q&A interface and clicks Submit.

Preconditions

Student is authenticated. A course context has already been selected.

Postconditions

The question is accepted by the system and passed into the response-generation process.

Normal Flow

1. Student opens the Q&A interface.
2. Student types a natural language question.
3. Student submits the question.
4. System validates the Student session and request.
5. System processes the question within the selected course context.
6. System forwards the request for retrieval and answer generation.

Alternative Flows

1a. If the Student submits an empty question, system prompts the Student to enter valid text.

2a. If the question exceeds the allowed input length, system displays a validation error and prevents submission.

UC-03 Receive AI-Generated Response

Description

A Student receives an AI-generated answer based on relevant course materials retrieved by the system.

Actor

Student (Primary)

Trigger

System finishes processing the Student's submitted question.

Preconditions

Student has submitted a valid question. Relevant course materials are indexed in the selected course knowledge base.

Postconditions

A grounded answer is displayed to the Student and stored in the current conversation session.

Normal Flow

1. System retrieves relevant material from the selected course knowledge base.
2. System generates an answer using the retrieved material.
3. System displays the answer to the Student.
4. System attaches source citations to the answer.
5. System stores the interaction in the conversation history.

Alternative Flows

1a. If no relevant material is found, system displays a fallback message explaining that no grounded answer could be generated.

2a. If answer generation fails, system displays an error notice and asks the Student to try again later.

UC-04 View Source References

Description

A Student opens a citation attached to an answer to inspect the original supporting material.

Actor

Student (Primary)

Trigger

Student clicks a source citation shown with an answer.

Preconditions

Student has received an answer containing at least one citation.

Postconditions

Student can inspect the cited source material used to support the answer.

Normal Flow

1. Student reads the answer displayed by the system.
2. Student clicks a source citation.
3. System retrieves the referenced source information.
4. System displays the document name, page number, and relevant excerpt.
5. Student reviews the source to verify the answer.

Alternative Flows

1a. If the source content is unavailable, system displays a message that the reference cannot be opened.

2a. If the citation metadata is incomplete, system displays the available source information only.

UC-05 Continue Multi-turn Conversation

Description

A Student asks follow-up questions within the same session so that the system can preserve short conversational context.

Actor

Student (Primary)

Trigger

Student enters a follow-up question in an existing conversation session.

Preconditions

Student is authenticated. An active conversation session exists.

Postconditions

The conversation continues in the same session and the new interaction is saved.

Normal Flow

1. Student opens an existing conversation or remains in the current session.
2. Student reviews the previous answer.
3. Student enters a follow-up question.
4. System loads the recent session history.
5. System uses the recent context together with the new question.
6. System generates and displays the next answer.
7. System stores the new interaction in the same conversation thread.

Alternative Flows

1a. If the session has expired, system prompts the Student to start a new session.

2a. If the previous conversation history cannot be loaded, system continues with the new question only and notifies the Student.

UC-06 Resume Saved Session

Description

A Student reopens a previously saved conversation so that they can continue studying from earlier work.

Actor

Student (Primary)

Trigger

Student selects a saved session from the session history panel.

Preconditions

Student is authenticated. At least one saved session exists.

Postconditions

The selected session is restored and available for continued interaction.

Normal Flow

1. Student opens the session history panel.
2. System displays previously saved sessions.
3. Student selects one saved session.
4. System loads the conversation history for that session.
5. Student reviews earlier questions and answers.
6. Student continues the session if needed.

Alternative Flows

1a. If no saved sessions exist, system displays an empty-state message and prompts the Student to start a new session.

2a. If the selected session cannot be loaded, system displays an error and returns the Student to the session list.

UC-07 Review Conversation History

Description

A Student reviews previous questions and answers from an active or saved conversation session for study and revision purposes.

Actor

Student (Primary)

Trigger

Student opens an active or saved conversation session.

Preconditions

Student is authenticated. A conversation session exists.

Postconditions

Student can review earlier questions, answers, and citations in the selected session.

Normal Flow

1. Student opens the current or saved conversation session.
2. System displays the sequence of previous questions and answers.
3. Student scrolls through the conversation history.
4. Student reviews earlier explanations and citations for revision.

Alternative Flows

1a. If the selected session contains no previous messages, system displays a message indicating that no conversation history is available.

UC-08 View Confidence Indicator

Description

A Student checks the confidence indicator attached to an answer to judge how reliable the response may be.

Actor

Student (Primary)

Trigger

System displays an answer with an associated confidence indicator.

Preconditions

Student has received an answer that includes a confidence indicator.

Postconditions

Student gains an additional signal to help judge whether the answer should be trusted or verified further.

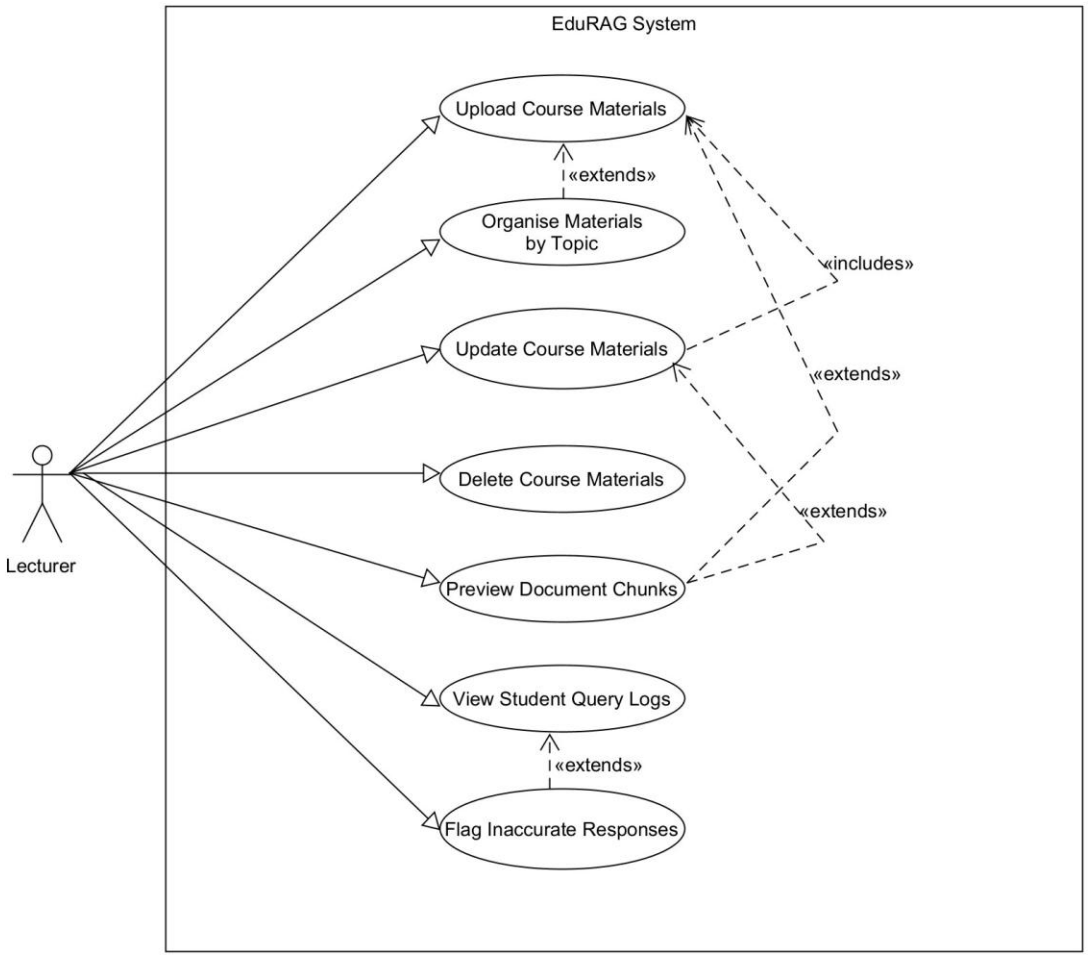
Normal Flow

1. Student submits a question and receives an answer.
2. System displays a confidence indicator alongside the answer.
3. Student reviews the confidence level.
4. Student uses the confidence level to decide whether to trust the answer or inspect citations more carefully.

Alternative Flows

1a. If confidence scoring is unavailable, system displays a notice that confidence could not be calculated for the response.

Lecturer Use Cases



ID	Use case	Description
UC-09	Upload course materials	Lecturer uploads new course files for ingestion into the knowledge base.
UC-10	Organise materials by topic	Lecturer applies topic or week tags to improve content organisation.
UC-11	Update course materials	Lecturer replaces an existing document with a newer version.
UC-12	Delete course materials	Lecturer removes outdated or incorrect content and its related embeddings.
UC-13	Preview document chunks	Lecturer checks chunking output to validate extraction quality.
UC-14	View student query logs	Lecturer reviews query patterns to identify confusion points in course content.
UC-15	Flag inaccurate responses	Lecturer reports suspicious answers for later review.

UC-09 Upload Course Materials

Description

A Lecturer uploads PDF, PPTX, or DOCX course materials to the EduRAG system so that students can query them through the course knowledge base.

Actor

Lecturer (Primary)

Trigger

Lecturer navigates to Content Management and selects Upload Material.

Preconditions

Lecturer is authenticated. Lecturer is assigned to the target course. File is in a supported format and within the allowed size limit.

Postconditions

The uploaded document is stored, queued for indexing, and becomes retrievable after successful processing.

Normal Flow

1. Lecturer navigates to the Content Management section.
2. Lecturer selects Upload Material and chooses a file.
3. System validates file type and size.
4. System stores the file and starts text extraction, chunking, and embedding generation.
5. System indexes the document data in ChromaDB using the correct course_id metadata.
6. System confirms that the upload has been accepted and shows indexing status information.

Alternative Flows

1a. If file type is unsupported, system displays an error and prompts the Lecturer to upload a valid file type.

2a. If the file exceeds the maximum size, system rejects the upload and displays the allowed size limit.

3a. If indexing fails after upload, system marks the document as failed and shows a retry option.

UC-10 Organise Materials by Topic

Description

A Lecturer assigns topic tags or categories to uploaded documents so that retrieval can be filtered more precisely for student questions.

Actor

Lecturer (Primary)

Trigger

Lecturer selects one or more uploaded documents and opens the topic-tagging controls.

Preconditions

Lecturer is authenticated. At least one document has already been uploaded for the course.

Postconditions

The selected documents are tagged and can be used in topic-aware retrieval workflows.

Normal Flow

1. Lecturer navigates to the Content Management section and views the uploaded documents list.
2. Lecturer selects one or more documents.
3. Lecturer assigns topic tags from an existing list or creates new tags.
4. System stores the tag associations in document metadata.
5. System confirms the tag update with a success notification.
6. Tagged documents become available for topic-scoped retrieval.

Alternative Flows

1a. If the Lecturer enters a duplicate tag name, system suggests using the existing tag.

2a. If the Lecturer exceeds the allowed number of tags per document, system displays the limit and prevents the excess tag from being saved.

UC-11 Update Course Materials

Description

A Lecturer replaces an existing document with a newer version so that the knowledge base stays aligned with the latest teaching material.

Actor

Lecturer (Primary)

Trigger

Lecturer selects Replace for an existing document in Content Management.

Preconditions

Lecturer is authenticated. The original document exists in the system and belongs to the Lecturer's assigned course.

Postconditions

The new document version becomes active and the old indexed content is removed or archived.

Normal Flow

1. Lecturer navigates to the Content Management section and locates the existing document.
2. Lecturer selects Replace and uploads the new version.
3. System validates the replacement file.
4. System extracts text, chunks the content, and generates updated embeddings.
5. System removes the old embeddings from ChromaDB for the replaced document.
6. System indexes the new content under the same document record.
7. System confirms that the replacement was completed successfully.

Alternative Flows

1a. If re-indexing fails, system notifies the Lecturer and retains the original version as the active document.

2a. If the replacement file is invalid, system rejects the update and keeps the existing version unchanged.

UC-12 Delete Course Materials

Description

A Lecturer permanently removes an outdated or incorrect document from the EduRAG knowledge base so that students no longer receive answers grounded in stale content.

Actor

Lecturer (Primary)

Trigger

Lecturer clicks Delete on a selected document in Content Management.

Preconditions

Lecturer is authenticated. The target document exists and belongs to the Lecturer's assigned course. No conflicting indexing job is running for the document.

Postconditions

The document and its associated embeddings are removed from the active knowledge base and the deletion is recorded in the audit log.

Normal Flow

1. Lecturer navigates to Content Management and locates the document to delete.
2. Lecturer clicks Delete on the selected document.
3. System displays a confirmation dialog explaining that the file and its indexed content will be removed.
4. Lecturer confirms the deletion.
5. System removes the document file, metadata, and associated embeddings from ChromaDB.
6. System logs the deletion event and displays a success confirmation message.

Alternative Flows

1a. If the Lecturer cancels the confirmation dialog, no changes are made.

2a. If the deletion fails because of a system error or active background job, system displays an error and retains the document intact.

UC-13 Preview Document Chunks

Description

A Lecturer reviews how a document has been split into chunks so that extraction and indexing quality can be checked before relying on retrieval output.

Actor

Lecturer (Primary)

Trigger

Lecturer selects Preview Chunks for an uploaded document.

Preconditions

Lecturer is authenticated. The selected document has been processed successfully.

Postconditions

Lecturer can inspect chunk boundaries and decide whether the document is suitable for retrieval as indexed.

Normal Flow

1. Lecturer opens the uploaded documents list.
2. Lecturer selects a processed document.
3. Lecturer clicks Preview Chunks.
4. System retrieves the stored chunks for the selected document.
5. System displays chunk text, chunk order, and basic metadata.
6. Lecturer reviews the chunking output for quality and completeness.

Alternative Flows

1a. If the document has not finished processing, system displays a message that chunk preview is not yet available.

2a. If no chunks were generated, system displays a processing warning and suggests re-uploading or replacing the document.

UC-14 View Student Query Logs

Description

A Lecturer reviews student query logs and usage patterns for their course so that common confusion points and knowledge gaps can be identified.

Actor

Lecturer (Primary)

Trigger

Lecturer opens the query log or analytics view for a course.

Preconditions

Lecturer is authenticated. Lecturer has access to the selected course. Query logging is enabled.

Postconditions

Lecturer has visibility into common question patterns and can use that information to refine teaching material or document coverage.

Normal Flow

1. Lecturer navigates to the course analytics or query log page.
2. System displays recent student queries and related usage metrics for that course.
3. Lecturer filters the logs by date range, topic, or document.
4. Lecturer reviews repeated questions and areas with frequent query activity.
5. System allows the Lecturer to inspect selected query records in more detail.

Alternative Flows

1a. If no logs exist for the selected period, system displays a No data available message.

2a. If the Lecturer attempts to access logs for an unassigned course, system denies access and shows an authorisation message.

UC-15 Flag Inaccurate Responses

Description

A Lecturer flags a response that appears inaccurate, misleading, or poorly grounded so that it can be reviewed by the team or administrators.

Actor

Lecturer (Primary)

Trigger

Lecturer selects a flag or report option on a questionable system response.

Preconditions

Lecturer is authenticated. A response record exists for the selected interaction.

Postconditions

The flagged response is recorded for follow-up review and linked to the relevant course and conversation context.

Normal Flow

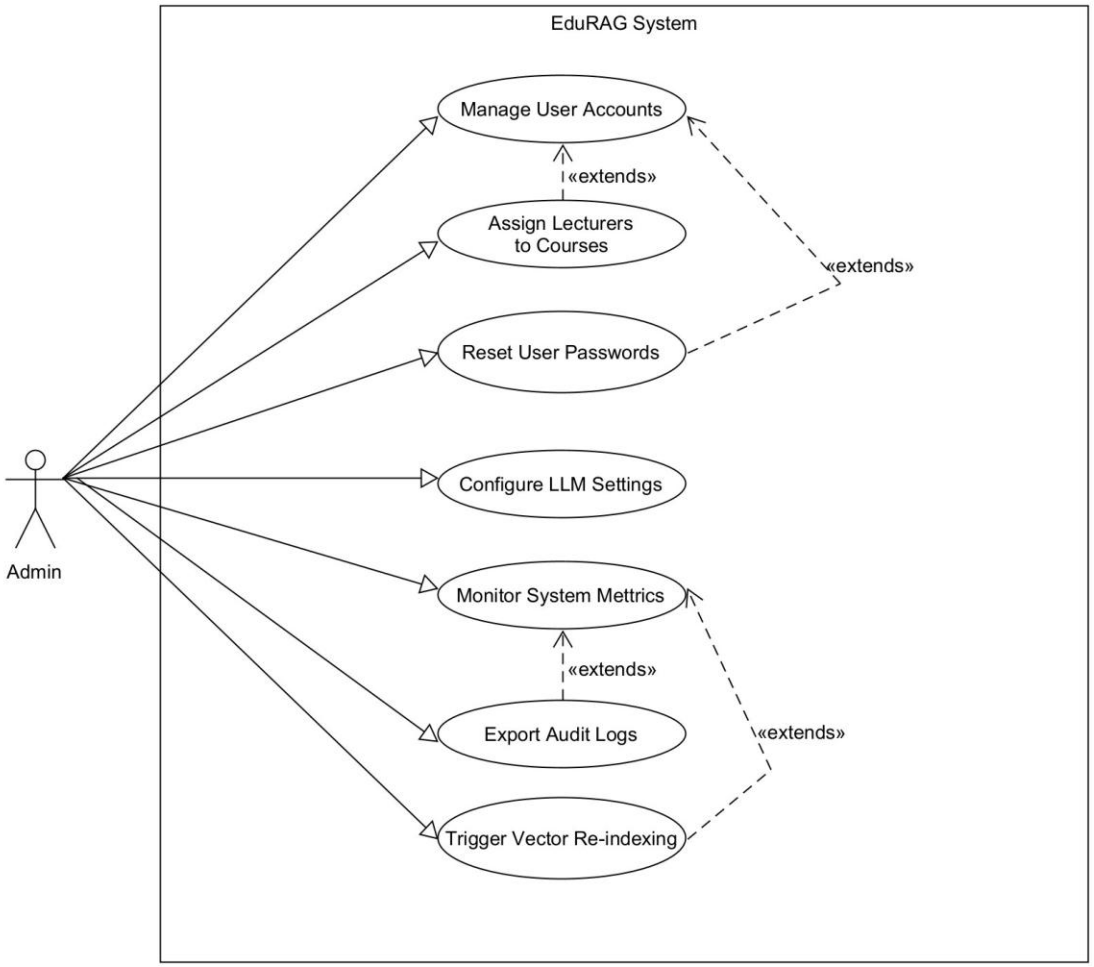
1. Lecturer reviews a system response.
2. Lecturer selects the flag or report action.
3. System prompts the Lecturer to provide a reason or category for the issue.
4. Lecturer submits the report.
5. System records the flagged response, associated metadata, and Lecturer comments.
6. System confirms that the issue has been logged for review.

Alternative Flows

1a. If the Lecturer cancels the report before submission, system discards the draft and makes no change.

2a. If the response record cannot be found, system displays an error and prevents the flagging action.

Admin Use Cases



ID	Use case	Description
UC-16	Manage user accounts	Admin creates, updates, or deactivates Student and Lecturer accounts.

UC-17	Assign lecturers to courses	Admin links lecturers to the correct course ownership scope.
UC-18	Reset user passwords	Admin assists users who need credential recovery.
UC-19	Configure LLM settings	Admin manages provider, key, and model configuration.
UC-20	Monitor system metrics	Admin reviews usage and performance indicators.
UC-21	Export audit logs	Admin accesses audit records for traceability and compliance.
UC-22	Trigger vector re-indexing	Admin refreshes vector data after large content changes.

UC-16 Manage User Accounts

Description

An Admin creates, edits, activates, or deactivates user accounts for Lecturers and Students within the EduRAG platform.

Actor

Admin (Primary)

Trigger

Admin opens the User Management area and creates or edits a user record.

Preconditions

Admin is authenticated with Admin-level privileges.

Postconditions

The selected user account is created or updated correctly and the action is recorded in the audit log.

Normal Flow

1. Admin navigates to User Management.
2. Admin selects Create New User or opens an existing user record.
3. Admin enters or edits user details such as name, email, and role.
4. System validates the supplied information.
5. System saves the user record.
6. System sends any required welcome or update notification.
7. System confirms successful account creation or update and displays the updated user record in the list.

Alternative Flows

1a. If the email already exists, system prompts the Admin to resolve the duplicate before saving.

2a. If the Admin selects Deactivate, system suspends login access while retaining user data.

UC-17 Assign Lecturers to Courses

Description

An Admin assigns a Lecturer to one or more courses so that content ownership and permissions are scoped correctly.

Actor

Admin (Primary)

Trigger

Admin opens a course assignment control for a Lecturer or course.

Preconditions

Admin is authenticated. The Lecturer account and course record both exist.

Postconditions

The Lecturer is linked to the selected course and gains the correct course-level permissions.

Normal Flow

1. Admin navigates to the course assignment interface.
2. Admin selects a Lecturer account.
3. Admin selects one or more courses to assign.
4. System validates that the assignment is permitted.
5. System stores the course assignment mapping.
6. System confirms the assignment and updates the Lecturer's course access.

Alternative Flows

1a. If the Lecturer is already assigned to the selected course, system displays a message that the mapping already exists.

2a. If the target course does not exist or is inactive, system prevents the assignment and displays an error.

UC-18 Reset User Passwords

Description

An Admin resets a password for a user who is locked out or requires credential recovery.

Actor

Admin (Primary)

Trigger

Admin selects Reset Password for a user account.

Preconditions

Admin is authenticated. The target user account exists.

Postconditions

A password reset is issued and the action is recorded for audit purposes.

Normal Flow

1. Admin navigates to User Management.

2. Admin searches for the target user account.
3. Admin selects Reset Password.
4. System generates a reset workflow or temporary credential according to system policy.
5. System sends the password reset instruction to the user or displays the temporary reset result to the Admin.
6. System records the reset action in the audit log.

Alternative Flows

1a. If the target account is inactive, system warns the Admin before proceeding.

2a. If the reset notification cannot be delivered, system logs the failure and prompts the Admin to retry or use another recovery path.

UC-19 Configure LLM Settings

Description

An Admin configures the model, provider, API key, and response parameters used by the EduRAG response generation module.

Actor

Admin (Primary)

Trigger

Admin opens the LLM Configuration panel in System Settings.

Preconditions

Admin is authenticated. A valid provider configuration or API key is available.

Postconditions

The approved model configuration is stored securely and becomes active for subsequent query responses.

Normal Flow

1. Admin navigates to System Settings.
2. Admin selects the LLM Configuration panel.

3. Admin enters or updates the provider, model, API key, and response parameters.
4. System validates the configuration by performing a test request or connection check.
5. System stores the approved settings securely.
6. System confirms that the updated configuration is active.

Alternative Flows

1a. If API key validation fails, system alerts the Admin and does not save the invalid configuration.

2a. If the selected model is unavailable for the provider, system prompts the Admin to choose another configuration.

UC-20 Monitor System Metrics

Description

An Admin views a monitoring dashboard showing usage statistics, response time metrics, and system health indicators.

Actor

Admin (Primary)

Trigger

Admin navigates to the Monitoring Dashboard.

Preconditions

Admin is authenticated. System is online and metrics collection is enabled.

Postconditions

Admin has reviewed system health and any detected performance issues are available for follow-up.

Normal Flow

1. Admin navigates to the Monitoring Dashboard.
2. System displays daily or weekly query counts, average response times, and active users.
3. Admin filters metrics by date range or course.
4. Admin reviews flagged alerts if any thresholds have been exceeded.

5. System updates the dashboard view based on the selected filters.

Alternative Flows

1a. If no data exists for the selected range, system shows a No data available message.

2a. If the monitoring service is temporarily unavailable, system displays an error and provides the latest cached metrics if available.

UC-21 Export Audit Logs

Description

An Admin views and exports audit logs for system activities such as user logins, document uploads, deletions, and indexing actions.

Actor

Admin (Primary)

Trigger

Admin opens the audit log view and selects an export action.

Preconditions

Admin is authenticated. Audit logging is enabled and log data exists for the selected period.

Postconditions

The requested audit data is displayed or exported for compliance, review, or investigation purposes.

Normal Flow

1. Admin navigates to the Audit Logs page.
2. System displays available audit records with filtering controls.
3. Admin filters logs by date, actor, event type, or course.
4. Admin reviews the filtered audit records.
5. Admin selects Export Logs.

6. System generates and downloads the export file in the supported format.

Alternative Flows

1a. If no audit records match the selected filters, system displays a No matching logs message.

2a. If export generation fails, system displays an error and keeps the filtered logs visible for manual review.

UC-22 Trigger Vector Re-indexing

Description

An Admin triggers a re-indexing process after bulk content changes so that the vector store remains aligned with the latest course materials.

Actor

Admin (Primary)

Trigger

Admin opens the maintenance controls and selects a re-index action.

Preconditions

Admin is authenticated. Content exists for the selected course or indexing scope.

Postconditions

A re-indexing job is started or scheduled and the system tracks its status until completion.

Normal Flow

1. Admin navigates to the maintenance or indexing controls.
2. Admin selects the course or content scope to re-index.
3. Admin confirms the re-index action.
4. System queues the re-indexing job.
5. System regenerates embeddings and refreshes the relevant ChromaDB records.
6. System records the operation in the audit log and reports job status to the Admin.

Alternative Flows

1a. If a re-index job is already running for the selected scope, system prevents a duplicate job and shows the current status instead.

2a. If the re-index operation fails, system marks the job as failed and provides diagnostic information.

Appendix F. Traceability Matrix

FR ID	Module(s)	Source User Story	Notes
FR-001	M4	US-A1, US-S1, US-L1	Foundation for all user-facing features
FR-002	M4	US-A1, US-S1, US-L1	Enables Lecturer and Admin roles
FR-003	M1, M4	US-L1	Primary document ingestion path
FR-004	M1	US-L1	Prerequisite for chunking
FR-005	M1	US-L1, US-L4	Enables embedding stage
FR-006	M2	US-L1	Prerequisite for semantic search
FR-007	M2	US-L1, US-S5	Enables multi-course scoping
FR-008	M2	US-S1, US-S5	Core RAG retrieval step
FR-009	M4	US-S1	Primary user interaction
FR-010	M3	US-S1, US-S2	Core RAG generation step
FR-011	M3, M4	US-S2	Trust and verification for users

FR-012	M1, M4	US-L1, US-L4	Required by 100 MB document size target
FR-013	M2	US-S5, US-A2	Real-world course isolation
FR-014	M3, M4	US-S1	Baseline Q&A mode
FR-015	M4	US-A6	Supports audit and debugging
FR-016	M2, M3	US-S7	Graceful handling of edge cases
FR-017	All	(security foundation)	Derived from security NFRs
FR-018	M4	US-A1	Derived from NFR-011
FR-019	M1	US-L1	Broader format coverage
FR-020	M1	US-L1	Broader format coverage
FR-021	M3, M4	US-S3	Enables follow-up questions
FR-022	M4	US-S4, US-S8	Persistent conversation history
FR-023	M4	US-A5	Admin feature
FR-024	M4	US-A6	Admin feature
FR-025	M4	(platform protection)	Derived from NFR-015
FR-026	M4	US-L1, US-L3	Lecturer content management
FR-027	M2, M3, M4	US-S6	Confidence indicator
FR-028	M2	US-L6, US-A5	Derived from NFR-005, NFR-006
FR-029	M4	US-S8	Export feature

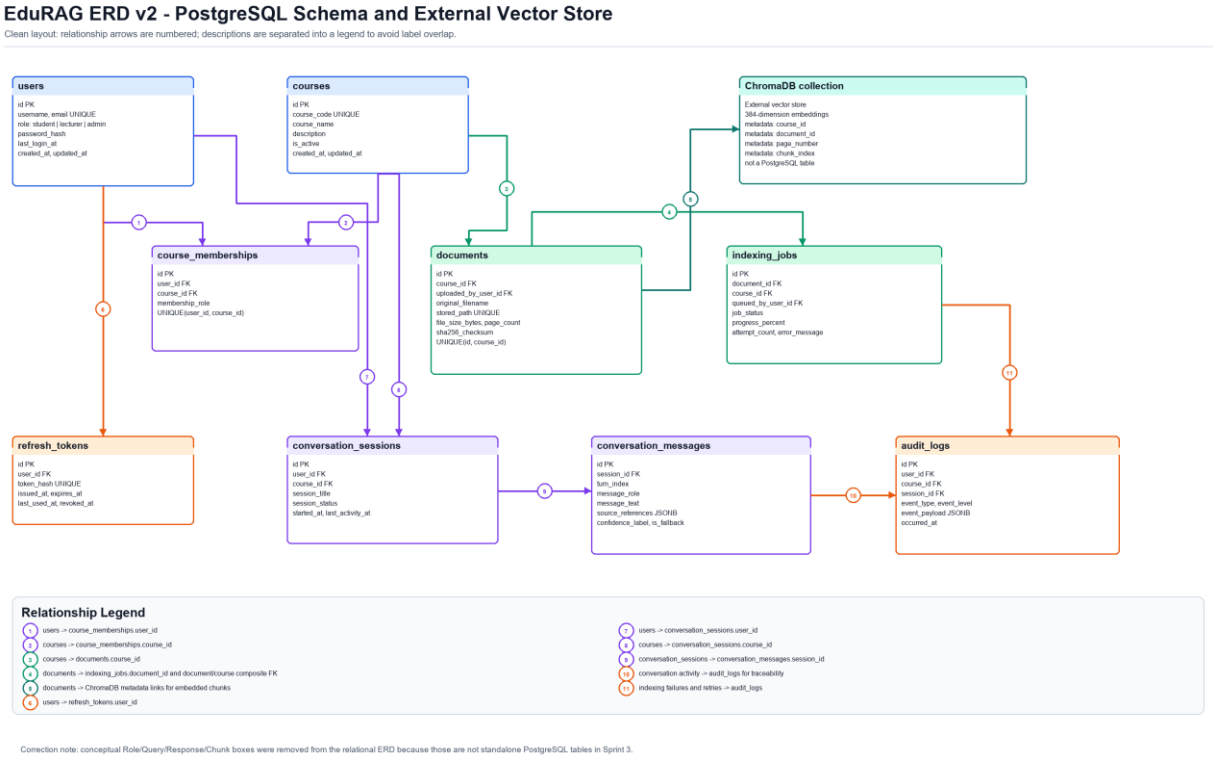
FR-030	M3	US-S8	Summary feature
FR-031	M3, M4	US-S3	Enhanced UX
FR-032	M4	(UI enhancement)	Cosmetic enhancement
FR-033	M4	US-A5	Admin feature

Appendix G. Additional Diagrams

This appendix presents one clear version of each key diagram type.

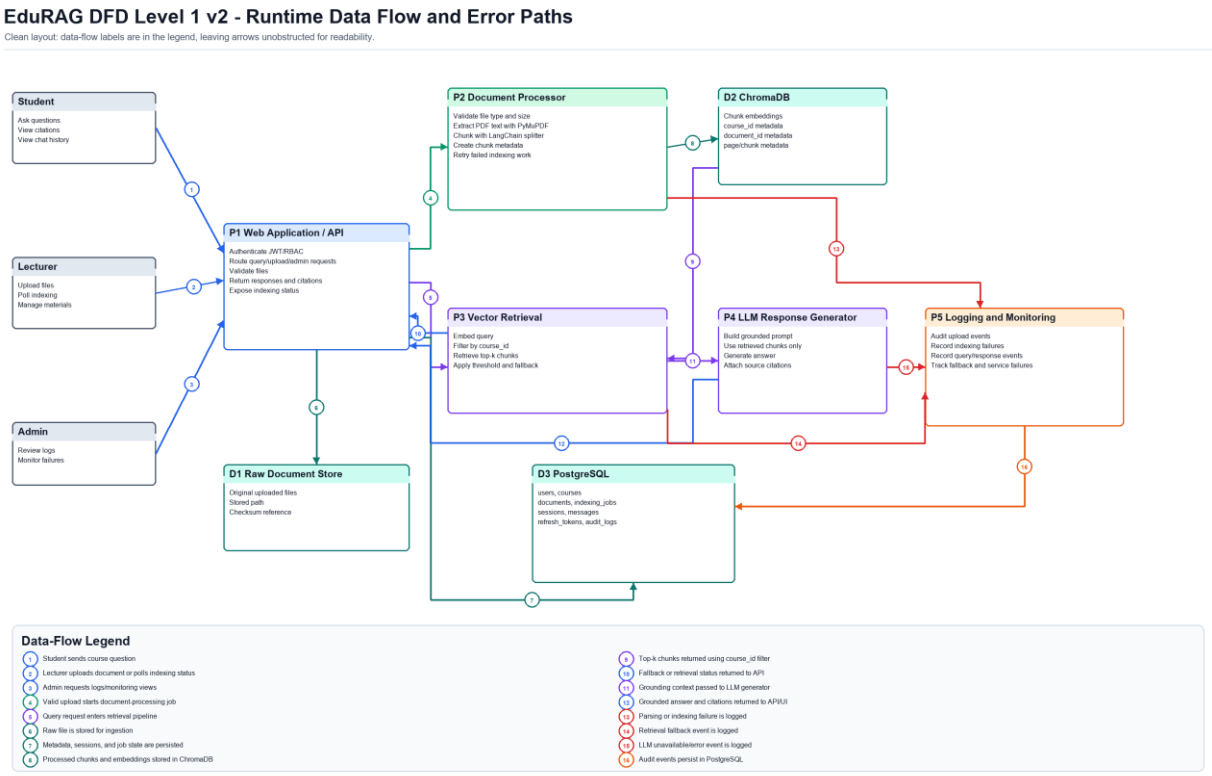
G1. Entity Relationship Diagram (ERD)

Function: defines core relational entities and relationships for users, sessions, logs, and course assets. Technique used: normalized ERD mapping for relational schema design.



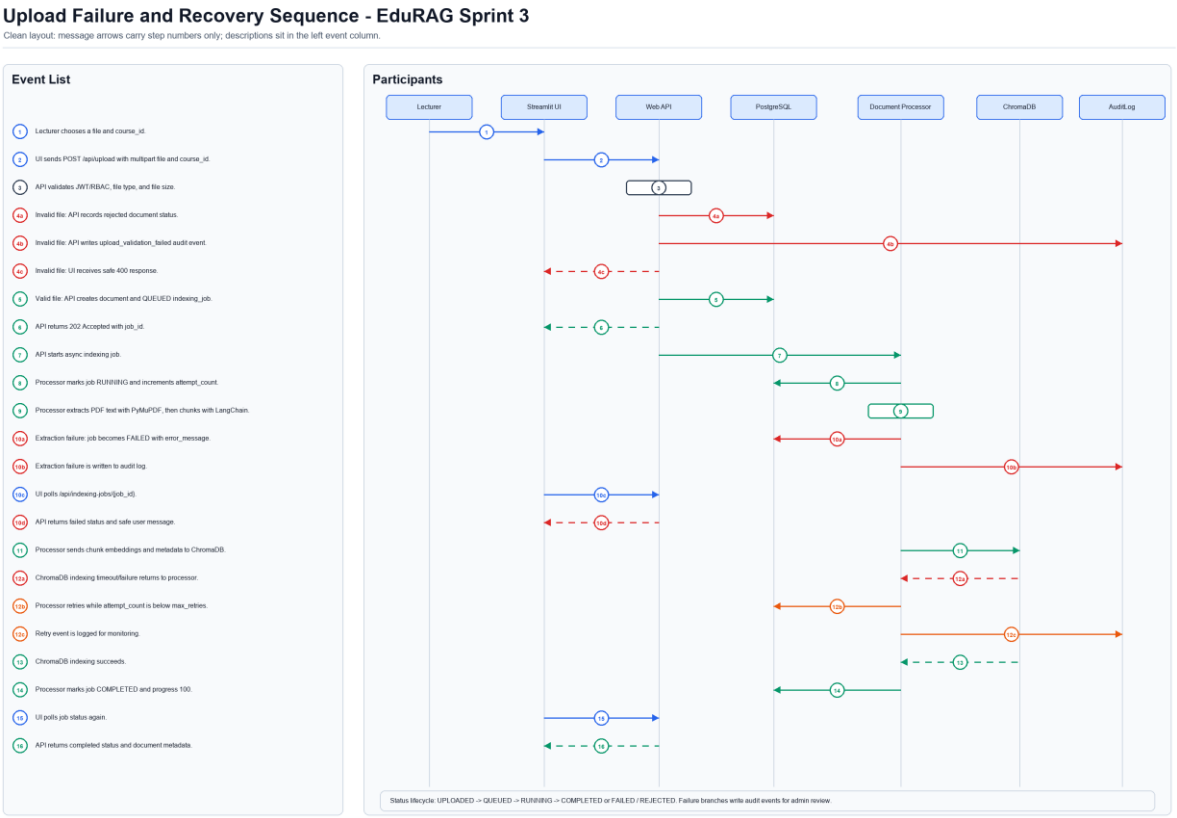
G2. Data Flow Diagram (DFD Level 1)

Function: shows module-level data movement across web application, document processing, retrieval, and response generation.
Technique used: Level 1 data-flow decomposition.



G3. Sequence Diagram (Upload exception and recovery)

Function: traces error-path orchestration for upload and indexing failure handling. Technique used: UML sequence diagram for operational recovery flow.

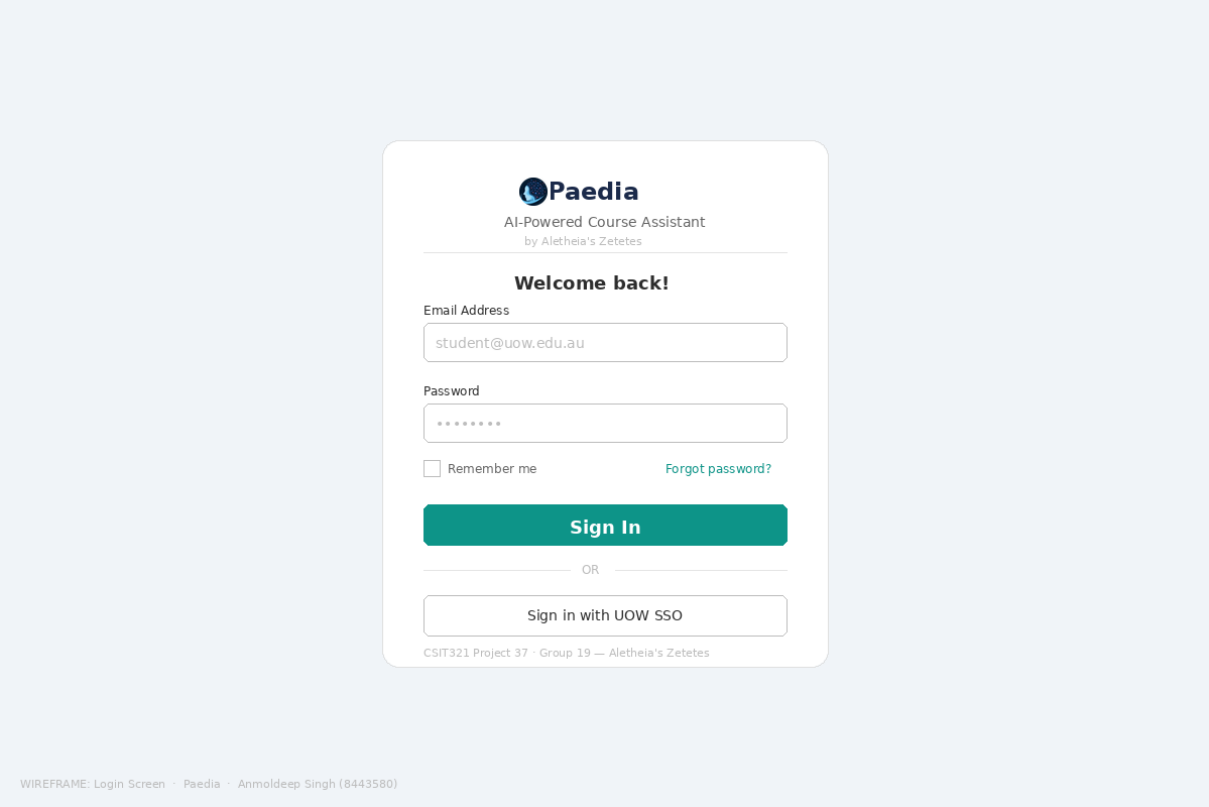


Appendix H. Initial Interface Wireframes

This appendix records the full Sprint 3 interface design set. The wireframes are included as design evidence, not as a final UI lock-in. They show how the current requirements can be realised through role-based navigation, course-scoped Q&A, upload and indexing status, citation visibility, confidence display, fallback labelling, and consistent visual treatment.

H1. Login and Role Entry

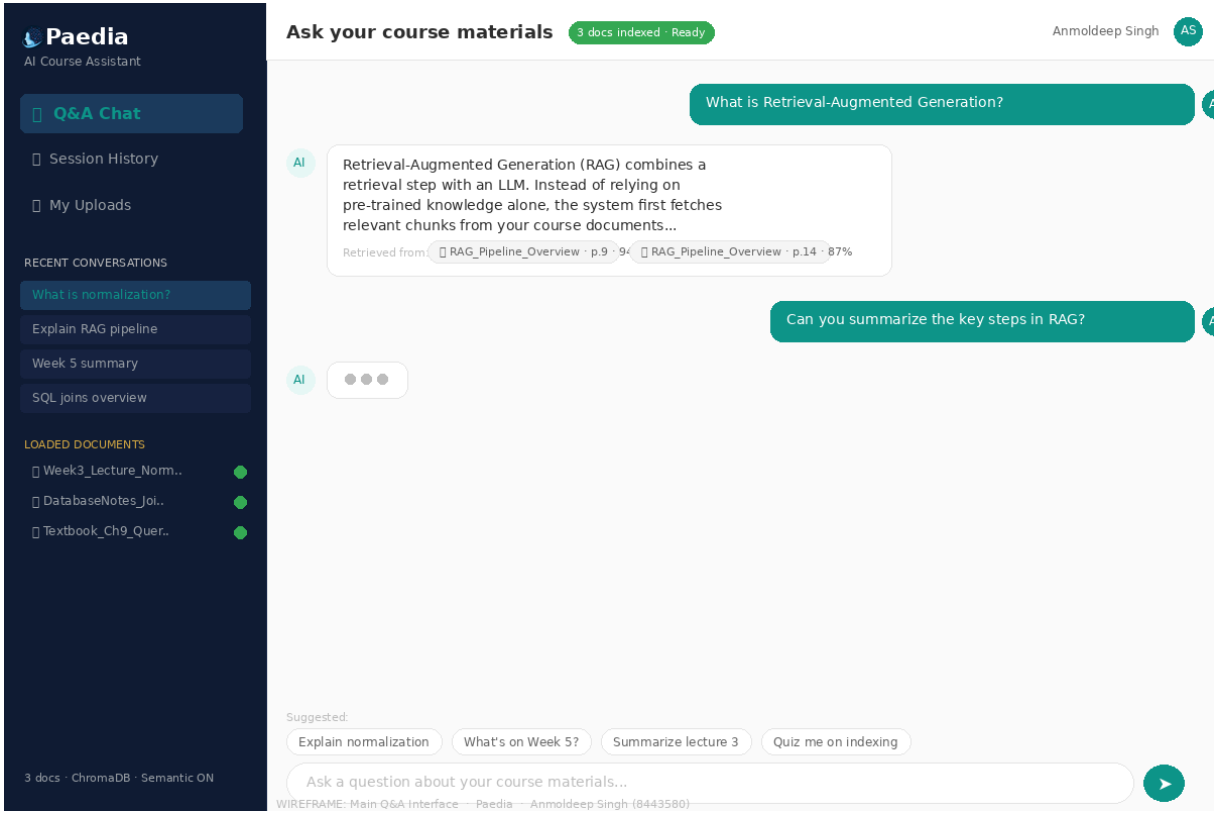
Shows product identity, role entry, and authentication entry point.



Wireframe H1. Shows product identity, role entry, and authentication entry point.

H2. Student Q&A Interface

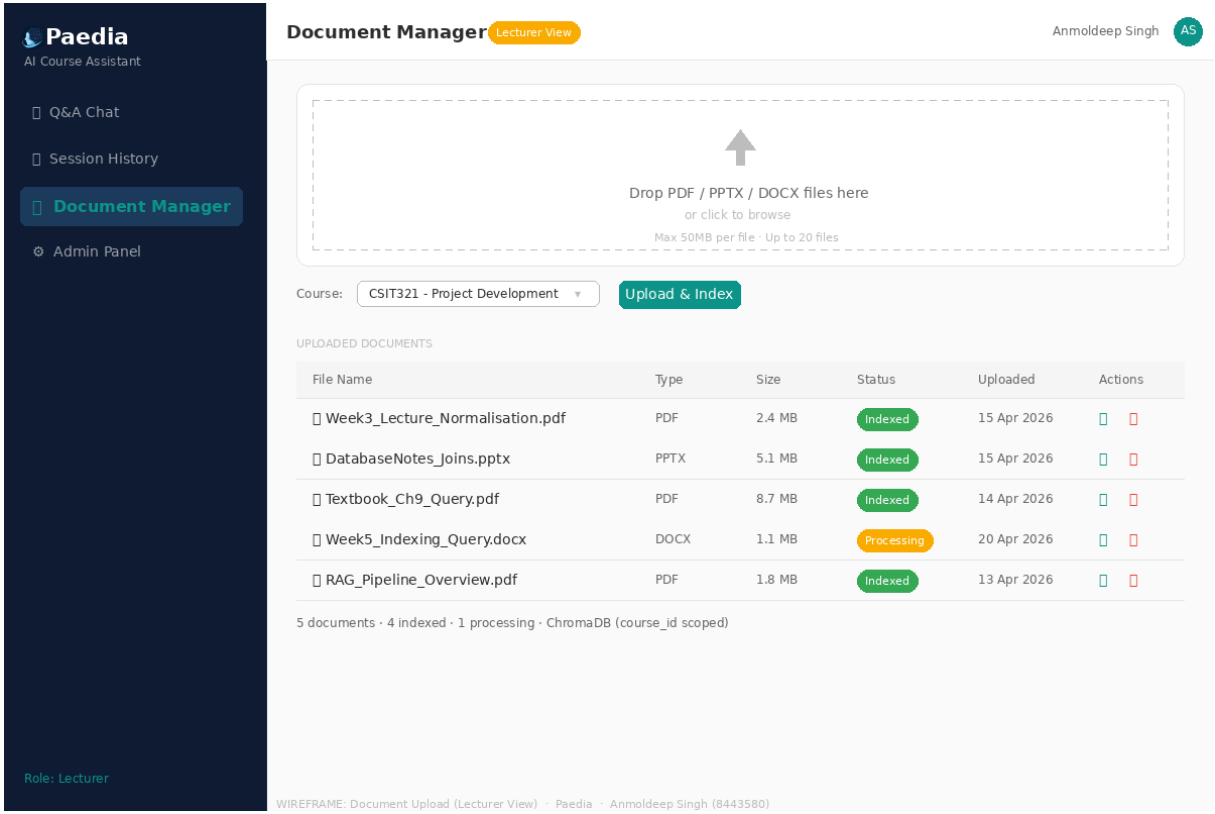
Shows course-scoped chat, source citations, confidence display, and suggested prompts.



Wireframe H2. Shows course-scoped chat, source citations, confidence display, and suggested prompts.

H3. Lecturer Document Upload

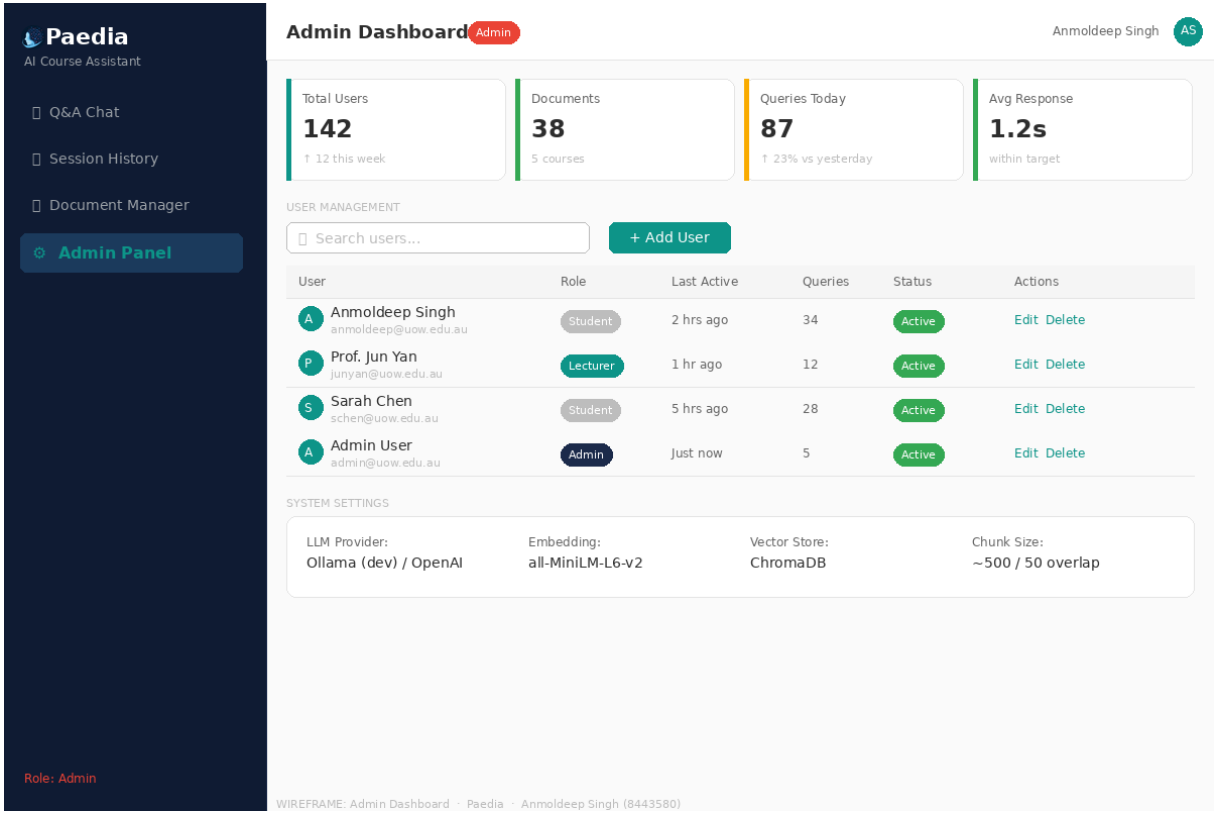
Shows upload, document-management, and indexing-status expectations for lecturer workflows.



Wireframe H3. Shows upload, document-management, and indexing-status expectations for lecturer workflows.

H4. Admin Dashboard

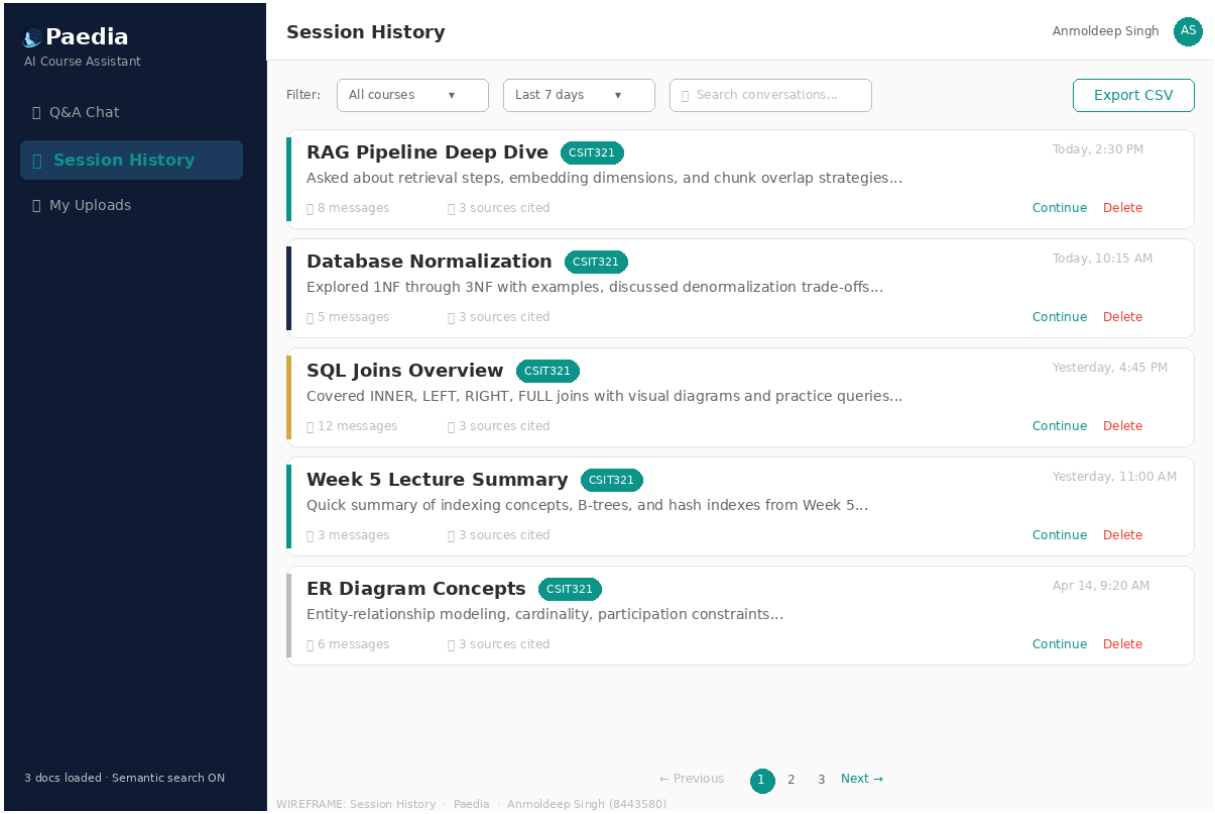
Shows role-aware monitoring, system metrics, and administrative oversight.



Wireframe H4. Shows role-aware monitoring, system metrics, and administrative oversight.

H5. Session History

Shows saved conversations and continuity for multi-turn learning sessions.



Wireframe H5. Shows saved conversations and continuity for multi-turn learning sessions.